INSTITUTO FEDERAL
Paraná
Campus Pinhais

INSTITUTO FEDERAL DO PARANÁ

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 3 – EBPF PARA REDES

PINHAIS
2026

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 3 – EBPF PARA REDES

Manual produzido no âmbito do projeto de extensão do minicurso Introdução ao eBPF, apresentado à disciplina de Práticas de Extensão V, do curso de Bacharelado em Ciência da Computação do Instituto Federal do Paraná, Campus Pinhais, sob a orientação do professor Guilherme Werneck de Oliveira.

Orientador: Prof. Guilherme Werneck de Oliveira

PINHAIS
2026

SUMÁRIO

1 INTRODUÇÃO **4**
1.1 EBPF APLICADO A REDES 4
1.2 VANTAGENS DO EBPF PARA REDES 5
2 PRÉ-REQUISITOS **7**
2.1 CONHECIMENTOS NECESSÁRIOS 7
2.2 AMBIENTE LINUX 8
2.3 CONFIGURAÇÃO DO AMBIENTE EBPF 8
3 FUNDAMENTOS DE EBPF **11**
3.1 ARQUITETURA DE REDE DO LINUX 11
3.2 KERNEL NETWORKING STACK 12
3.3 HOOKS DE REDE 12
3.4 EBPF PROGRAMS 13
3.5 EBPF MAPS 15
4 XDP — *EXPRESS DATA PATH* **16**
4.1 MODOS DE EXECUÇÃO 16
4.2 XDP E PROCESSAMENTO DE PACOTES 17
4.3 REDIRECIONAMENTO DE PACOTES 19
4.4 VANTAGENS E LIMITAÇÕES 20
5 EBPF E TRAFFIC CONTROL **22**
5.1 TC INGRESS 22
5.2 TC EGRESS 25
5.3 MANIPULAÇÃO, CLASSIFICAÇÃO E CONTROLE DE TRÁFEGO 26
6 OBSERVABILIDADE DE REDE **30**
6.1 MONITORAMENTO DE INTERFACES, PACOTES E CONEXÕES 30
6.2 LATÊNCIA 32
6.3 THROUGHPUT 34
6.4 NETWORK FLOWS 36
7 SEGURANÇA DE REDE **39**
7.1 FILTRAGEM DE PACOTES 39
7.2 FIREWALL BASEADO EM EBPF 42
7.3 DETECÇÃO DE TRÁFEGO SUSPEITO 45
8 EBPF EM CONTAINERS E KUBERNETES **49**
8.1 NETWORKING EM CONTAINERS E KUBERNETES 49
8.2 CNI 51
8.3 NETWORK POLICIES 52
8.4 COMUNICAÇÃO ENTRE PODS E SERVIÇOS 53
9 CILIUM E EBPF **56**
9.1 CILIUM COMO CNI 56
9.2 NETWORK POLICIES 58
9.3 OBSERVABILIDADE COM HUBBLE 60
9.4 SEGURANÇA DE REDE COM CILIUM 60
10 FERRAMENTAS DE EBPF NETWORKING **62**
10.1 BCC 62
10.2 BPFTRACE 63
10.3 BPFTOOL 64
10.4 LIBBPF 65

11 EXEMPLOS PRÁTICOS **68**
11.1 CAPTURA E ANÁLISE DE PACOTES 68
11.2 MONITORAMENTO DE CONEXÕES 70
11.3 FILTRAGEM DE TRÁFEGO 72
12 CONCLUSÃO **74**

1 INTRODUÇÃO

O *extended Berkeley Packet Filter* (eBPF) é uma tecnologia do kernel Linux que permite a execução de programas verificados diretamente no espaço do kernel, sem necessidade de modificá-lo ou carregar módulos adicionais (Linux Kernel Documentation, 2024). Originada do BSD Packet Filter proposto por McCanne e Jacobson (1993) para captura eficiente de pacotes em nível de usuário, a tecnologia evoluiu para suportar casos de uso muito além da análise de rede: rastreamento de desempenho, observabilidade de sistemas e, de forma crescente, processamento de rede de alto desempenho (GREGG, 2019).

No contexto de redes, o eBPF assume relevância especial em ambientes de funções virtualizadas de rede (NFV), onde a capacidade de instrumentar e monitorar funções de rede sem impacto perceptível sobre o desempenho é um requisito crítico (ETSI Industry Specification Group (ISG) NFV, 2014). Alves e Plinta (2026) investigaram justamente esse cenário, comparando ferramentas de extração de comportamento de funções virtualizadas — trabalho que serviu de base para o minicurso documentado neste relatório. O repositório de código e os materiais práticos utilizados estão disponíveis em Plinta e Nahas (2026).

1.1 EBPF APLICADO A REDES

A aplicação do eBPF em redes se dá em múltiplas camadas da pilha de comunicação, desde o processamento de pacotes no nível do driver de rede até a instrumentação de aplicações de espaço de usuário que implementam protocolos (Linux Kernel Documentation, 2024).

***eXpress Data Path* (XDP).** O XDP é o ponto de intervenção mais precoce disponível para programas eBPF na pilha de rede do kernel Linux (Linux Kernel Documentation, 2024). Um programa XDP é executado diretamente no driver da interface de rede, antes que o pacote seja alocado em um buffer `sk_buff` e entregue à pilha de rede convencional. Isso permite descartar, redirecionar ou modificar pacotes com latência próxima à do hardware, viabilizando casos de uso como mitigação de DDoS, balanceamento de carga e encaminhamento acelerado (GREGG, 2019).

***Traffic Control* (TC).** O subsistema de controle de tráfego do kernel expõe *hooks* eBPF tanto na entrada quanto na saída de pacotes, após o processamento de camada 2 e com acesso ao contexto completo do `sk_buff` (Linux Kernel Documentation, 2024). Essa posição na pilha oferece mais informações sobre o pacote do que o XDP — incluindo metadados de cgroup e namespace — ao custo de execução mais tardia. O TC é amplamente utilizado em implementações de políticas de rede por identidade em clusters Kubernetes, como as do projeto Cilium (CASSAGNES *et al.*, 2020).

Rastreamento de sockets e chamadas de sistema de rede. Programas eBPF conecta-

dos a *tracepoints* de chamadas de sistema como `connect`, `accept`, `sendmsg` e `recvmsg` oferecem visibilidade sobre o comportamento de rede de cada processo individualmente, associando fluxos de rede a PIDs, UIDs e cgroups (GREGG, 2019). Essa granularidade é essencial em ambientes NFV, onde múltiplas funções de rede virtualizadas compartilham o mesmo host e precisam ser monitoradas de forma isolada (VENÂNCIO *et al.*, 2021).

Monitoramento passivo de latência TCP. O eBPF permite medir a latência de conexões TCP diretamente no kernel, sem modificar as aplicações ou gerar tráfego adicional de sonda (RESEARCH, 2024). Métricas como RTT (*round-trip time*), retransmissões e janela de congestionamento são acessíveis via *tracepoints* de socket e podem ser agregadas em histogramas por fluxo, endereço de destino ou cgroup, fornecendo uma visão de desempenho de rede de alta fidelidade (GREGG, 2020).

Exportação de métricas para sistemas de monitoramento. Ferramentas como o `ebpf_exporter` (Cloudflare, s.d.) traduzem métricas coletadas por programas eBPF para o formato Prometheus (Prometheus Authors, s.d.), permitindo sua visualização em dashboards Grafana (Grafana Labs, s.d.) e integração com pipelines de observabilidade já existentes. Essa cadeia — eBPF no kernel, Prometheus para coleta, Grafana para visualização — é o padrão adotado no minicurso para demonstrar o monitoramento de funções virtualizadas de rede (ALVES; PLINTA, 2026).

1.2 VANTAGENS DO EBPF PARA REDES

A adoção do eBPF em aplicações de rede é motivada por um conjunto de vantagens que o distinguem tanto de abordagens baseadas em espaço de usuário quanto de soluções que exigem modificação do kernel (CASSAGNES *et al.*, 2020).

Baixo overhead de monitoramento. Ferramentas tradicionais de monitoramento de rede — como `sysstat` (GODARD; Sysstat Contributors, 2024) e contadores do sistema de arquivos `/proc` (Linux man-pages project, 2024) — realizam coleta periódica (*polling*) com resolução tipicamente de um segundo, perdendo eventos de curta duração e introduzindo imprecisão temporal. Cassagnes *et al.* (2020) demonstraram que programas eBPF impõem sobrecarga significativamente menor do que ferramentas de monitoramento convencionais, tornando o monitoramento contínuo viável mesmo em enlaces de alta taxa de transferência.

Granularidade por fluxo e por processo. Enquanto contadores de interface de rede agregam todo o tráfego de um nó, programas eBPF no TC ou em *tracepoints* de socket associam cada pacote ou conexão ao processo, cgroup e namespace responsável (Linux Kernel Documentation, 2024). Em ambientes NFV, essa granularidade permite isolar o comportamento de rede de cada função virtualizada individualmente, sem necessidade de instrumentação da própria função (VENÂNCIO *et al.*, 2021).

Visibilidade sem modificação das aplicações. O eBPF opera como observador externo: não requer modificação do código-fonte das funções de rede monitoradas, não impõe dependências de biblioteca e não altera o comportamento observado (GREGG, 2019). Essa propriedade é especialmente valiosa em ambientes NFV com funções de terceiros, cujo código-fonte pode não estar disponível (ETSI Industry Specification Group (ISG) NFV, 2014).

Processamento no kernel sem chamadas de sistema adicionais. Ao executar diretamente no kernel, programas eBPF eliminam as trocas de contexto necessárias para comunicar eventos ao espaço de usuário em cada pacote ou conexão (Linux Kernel Documentation, 2024). Agregações — somas, médias, histogramas — são calculadas em mapas BPF no kernel e lidas pelo espaço de usuário em intervalos configuráveis, reduzindo drasticamente o volume de dados transmitidos entre kernel e aplicação de monitoramento (GREGG, 2020).

Programabilidade e extensibilidade. Ao contrário de contadores fixos do kernel ou de filtros estáticos de *firewall*, programas eBPF expressam lógica arbitrária — dentro das restrições do verificador — e podem ser atualizados dinamicamente sem interromper o tráfego de rede ou reiniciar serviços (Linux Kernel Documentation, 2024). Essa flexibilidade permite adaptar a instrumentação às necessidades específicas de cada função virtualizada sem ciclos de desenvolvimento no kernel (ALVES; PLINTA, 2026).

Portabilidade entre versões de kernel. O mecanismo BPF CO-RE (*Compile Once – Run Everywhere*) (NAKRYIKO, 2020), implementado pela libbpf (libbpf Contributors, s.d.), garante que um programa eBPF compilado uma única vez execute corretamente em diferentes versões do kernel Linux, sem recompilação. Em ambientes de produção com frotas heterogêneas de servidores — comum em infraestruturas NFV de grande escala — essa portabilidade reduz significativamente o custo operacional de manutenção de ferramentas de monitoramento (ETSI Industry Specification Group (ISG) NFV, 2014).

2 PRÉ-REQUISITOS

Este capítulo descreve os conhecimentos prévios recomendados e o ambiente necessário para acompanhar o minicurso e reproduzir os exemplos práticos de monitoramento de funções virtualizadas de rede com eBPF.

2.1 CONHECIMENTOS NECESSÁRIOS

O minicurso foi concebido para participantes com familiaridade básica com sistemas operacionais Linux, programação em linguagem C e fundamentos de redes de computadores. Não é exigida experiência prévia com desenvolvimento de kernel, com eBPF ou com funções virtualizadas de rede (NFV). Os tópicos a seguir representam a base mínima esperada.

Linha de comando Linux. O participante deve ser capaz de navegar pelo sistema de arquivos, executar comandos com privilégio de superusuário (`sudo`), instalar pacotes com o gerenciador de pacotes da distribuição e redirecionar a saída de programas para arquivos e para outros comandos via *pipe* (Linux man-pages project, 2024).

Linguagem C — nível introdutório. Programas eBPF são escritos em um subconjunto restrito de C e compilados para *bytecode* BPF (Linux Kernel Documentation, 2024). É suficiente compreender tipos primitivos, ponteiros, estruturas (`struct`) e a distinção entre espaço de usuário e espaço de kernel. Conceitos avançados, como alocação dinâmica de memória ou programação multithread, não são necessários neste nível introdutório.

Fundamentos de redes TCP/IP. Para os exemplos de monitoramento de rede, é necessário compreender o modelo de camadas TCP/IP, o conceito de pacote, como endereços IP e portas identificam conexões e a diferença entre protocolos orientados a conexão (TCP) e não orientados (UDP) (GREGG, 2019). Esse conhecimento contextualiza os pontos de observação que programas eBPF ocupam na pilha de rede do kernel.

Conceitos básicos de sistemas operacionais. É recomendável ter noção do que é uma chamada de sistema (*syscall*), como processos são escalonados e qual o papel do kernel na mediação entre aplicações e o hardware (GREGG, 2020). Em particular, compreender o que é um *file descriptor* e como o sistema de arquivos virtual `/proc` expõe informações de processos é útil para interpretar os dados coletados pelos programas eBPF (Linux man-pages project, 2024).

Noções de virtualização de funções de rede (NFV). Os exemplos práticos são contextualizados no cenário de monitoramento de funções virtualizadas de rede conforme o arcabouço do ETSI NFV (ETSI Industry Specification Group (ISG) NFV, 2014). Não é necessário ter experiência prévia com NFV, mas compreender que uma função de rede virtualizada é um software executando sobre infraestrutura de propósito geral — em vez de hardware dedicado — facilita a interpretação dos resultados obtidos (VENÂNCIO *et al.*, 2021).

2.2 AMBIENTE LINUX

Todos os exemplos foram desenvolvidos e testados em ambiente Linux com kernel versão 5.15 ou superior, que é o mínimo recomendado para suporte estável às funcionalidades de eBPF utilizadas no minicurso, incluindo `BPF_MAP_TYPE_RINGBUF` e suporte completo a BPF CO-RE (Linux Kernel Documentation, 2024). O código-fonte e os scripts de configuração do ambiente estão disponíveis no repositório público do minicurso (PLINTA; NAHAS, 2026).

Distribuição recomendada. Recomenda-se Ubuntu 22.04 LTS ou Debian 12, por oferecerem pacotes atualizados de ferramentas eBPF nos repositórios oficiais e ampla documentação de suporte. Outras distribuições baseadas em Debian ou em Red Hat (Fedora, RHEL 9+) são igualmente compatíveis, bastando adaptar os comandos de instalação de pacotes.

Uso em máquinas virtuais. O eBPF pode ser utilizado em máquinas virtuais, desde que o kernel do sistema convidado tenha sido compilado com as opções `CONFIG_BPF`, `CONFIG_BPF_SYSCALL` e `CONFIG_BPF_JIT` habilitadas (Linux Kernel Documentation, 2024). Para fins didáticos, recomenda-se o uso de uma máquina virtual com acesso irrestrito ao kernel, que oferece maior visibilidade sobre o sistema e evita restrições impostas por camadas de isolamento adicionais.

Verificação da versão do kernel. Antes de iniciar as atividades práticas, recomenda-se confirmar a versão do kernel em execução e verificar se o sistema de arquivos de depuração está montado:

Listing 2.1 – Verificação de versão do kernel e disponibilidade de tracepoints.

```
uname -r
ls /sys/kernel/debug/tracing/events/syscalls | head -5
```

Caso o diretório `/sys/kernel/debug` não exista ou esteja vazio, o sistema de arquivos de depuração deve ser montado manualmente:

Listing 2.2 – Montagem do sistema de arquivos de depuração.

```
sudo mount -t debugfs debugfs /sys/kernel/debug
```

2.3 CONFIGURAÇÃO DO AMBIENTE EBPF

Esta seção descreve a instalação das dependências necessárias para compilar e executar os programas eBPF do minicurso.

Instalação das dependências. Os seguintes pacotes devem estar presentes no sistema antes do início das atividades práticas:

- `clang` e `llvm` — compilador utilizado para gerar *bytecode* BPF a partir de código C (Linux Kernel Documentation, 2024);

- `libbpf-dev` — biblioteca de suporte a programas eBPF com suporte a CO-RE (libbpf Contributors, s.d.);
- `bpftrace` — linguagem de rastreamento de alto nível para investigação interativa (bpftrace Contributors, s.d.);
- `bpftool` — utilitário oficial para inspeção de programas, mapas e objetos BPF carregados no kernel (Linux Kernel Documentation, 2024);
- `linux-headers-$(uname -r)` — cabeçalhos do kernel necessários para compilação de programas eBPF;
- `make` e `gcc` — utilitários de construção (*build*).

Em distribuições baseadas em Debian, a instalação pode ser realizada com:

Listing 2.3 – Instalação das dependências em Ubuntu 22.04 / Debian 12.

```
sudo apt update
sudo apt install clang llvm libbpf-dev bpftrace bpftool \
                 linux-headers-$(uname -r) make gcc
```

Obtenção do repositório do minicurso. Os exemplos práticos estão organizados no repositório (PLINTA; NAHAS, 2026). Para cloná-lo e acessar o diretório de exemplos:

Listing 2.4 – Clonagem do repositório e acesso ao diretório de exemplos.

```
git clone https://github.com/joaopedroplinta/ebpf-scitec.git
cd ebpf-scitec
```

Verificação da instalação. Após instalar as dependências, recomenda-se executar um programa eBPF mínimo com o bpftrace para confirmar que o ambiente está funcional:

Listing 2.5 – Verificação do ambiente com bpftrace.

```
sudo bpftrace -e 'BEGIN { printf("eBPF funcionando!\n"); exit(); }'
```

A saída esperada é `eBPF funcionando!`, seguida do encerramento do processo. Caso ocorra erro de permissão, verifique se o comando está sendo executado com `sudo` ou se o usuário possui a capacidade `CAP_BPF` (Linux Kernel Documentation, 2024).

Instalação das ferramentas BCC. Para os exemplos que utilizam o BCC (IO Visor Project, s.d.), o pacote `bpfcc-tools` disponibiliza as ferramentas prontas para uso sem necessidade de compilação:

Listing 2.6 – Instalação das ferramentas BCC.

```
sudo apt install bpfcc-tools python3-bpfcc
```

Após a instalação, ferramentas como `execsnoop-bpfcc` e `opensnoop-bpfcc` podem ser executadas diretamente, e os scripts de monitoramento de rede — como `tcpconnect-bpfcc` e `tcpretrans-bpfcc` — estão disponíveis para uso imediato nos exemplos práticos do minicurso.

3 FUNDAMENTOS DE EBPF

Este capítulo apresenta os conceitos fundamentais necessários para compreender como o eBPF se integra à pilha de rede do Linux. O foco é construir um modelo mental claro da jornada de um pacote pelo kernel — e dos pontos em que programas eBPF podem observá-la ou modificá-la — antes de avançar para os exemplos práticos de monitoramento de funções virtualizadas de rede (Linux Kernel Documentation, 2024).

3.1 ARQUITETURA DE REDE DO LINUX

A pilha de rede do Linux é uma implementação em software do modelo TCP/IP, estruturada em camadas que separam responsabilidades e permitem que o kernel gerencie interfaces de rede heterogêneas com uma API uniforme para as aplicações (GREGG, 2020).

A jornada de um pacote recebido segue, de forma simplificada, as etapas abaixo:

1. **Driver da interface de rede** — o hardware sinaliza a chegada de um pacote via interrupção; o driver aloca um descritor de buffer (`sk_buff`) e copia os dados do pacote para a memória do sistema;
2. **Camada de enlace (L2)** — o kernel verifica o endereço MAC de destino e encaminha o quadro ao protocolo de rede correto (ARP, IPv4, IPv6);
3. **Camada de rede (L3)** — o subsistema IP realiza roteamento, fragmentação e verificação de integridade do cabeçalho;
4. **Camada de transporte (L4)** — TCP ou UDP desmultiplexa o pacote para o socket correspondente com base nas portas de origem e destino;
5. **Espaço de usuário** — a aplicação lê os dados do socket via chamadas de sistema como `recv(2)` ou `read(2)` (Linux man-pages project, 2024).

Na direção de saída, o caminho é percorrido no sentido inverso: a aplicação escreve no socket, o kernel empacota os dados, seleciona a rota, segmenta se necessário e entrega ao driver para transmissão (GREGG, 2019).

Cada etapa desse caminho expõe pontos de observação e intervenção para programas eBPF, que podem ser anexados em diferentes posições da pilha para coletar métricas, filtrar pacotes ou modificar cabeçalhos sem alterar o código das aplicações (Linux Kernel Documentation, 2024).

3.2 KERNEL NETWORKING STACK

O elemento central da pilha de rede do kernel Linux é o `sk_buff` (*socket buffer*), uma estrutura de dados que representa um pacote ao longo de toda a sua jornada pelo kernel (Linux Kernel Documentation, 2024). O `sk_buff` carrega não apenas os bytes do pacote, mas também metadados como a interface de entrada, marcações de QoS, estado de conexão e ponteiros para os cabeçalhos de cada camada — L2, L3 e L4 — permitindo que cada subsistema acesse diretamente a porção relevante do pacote sem recopiar os dados (GREGG, 2020).

Os subsistemas da pilha que interagem diretamente com programas eBPF incluem:

Netfilter. Framework de filtragem de pacotes sobre o qual *iptables* e *nftables* são construídos. Opera em cinco pontos de ancoragem (*hooks*) distribuídos ao longo do caminho de entrada e saída de pacotes (Linux Kernel Documentation, 2024). Embora o eBPF não opere diretamente sobre os *hooks* Netfilter, ambos coexistem na pilha e suas regras são avaliadas em sequência.

Controle de tráfego (*Traffic Control* — TC). Subsistema responsável pelo enfileiramento, escalonamento e modelagem de tráfego de saída, e pela classificação de tráfego de entrada (Linux Kernel Documentation, 2024). Expõe *hooks* eBPF tanto em entrada (`clsact ingress`) quanto em saída (`clsact egress`), com acesso completo ao `sk_buff` e aos metadados de cgroup e namespace do processo originador (CASSAGNES *et al.*, 2020).

Socket e camada de transporte. A camada de socket expõe *tracepoints* e *kprobes* sobre funções como `tcp_connect`, `tcp_close` e `udp_sendmsg`, permitindo rastrear o comportamento de rede de cada processo individualmente, com associação direta a PID, UID e cgroup (GREGG, 2019).

Fila de recepção multi-núcleo (RPS/RFS). Em sistemas com múltiplos núcleos de processamento, o kernel distribui o processamento de pacotes entre CPUs por meio dos mecanismos RPS e RFS (Linux Kernel Documentation, 2024). Programas eBPF do tipo `BPF_PROG_TYPE_SK_REUSEP` podem influenciar essa distribuição, balanceando fluxos entre workers de forma programável — caso de uso relevante em funções virtualizadas de rede que precisam escalar horizontalmente (VENÂNCIO *et al.*, 2021).

3.3 HOOKS DE REDE

O eBPF oferece pontos de ancoragem em múltiplas posições da pilha de rede, cada um com características distintas de latência, visibilidade e capacidade de intervenção (Linux Kernel Documentation, 2024). Os parágrafos a seguir apresentam os principais *hooks* e a posição de cada um no caminho de um pacote.

XDP (*eXpress Data Path*). O XDP é executado no driver da interface de rede, antes da

alocação do `sk_buff` (Linux Kernel Documentation, 2024). Por operar no ponto mais precoce da pilha, é o *hook* de menor latência disponível para programas eBPF — em implementações nativas (*native XDP*), o programa é executado diretamente no contexto de interrupção do driver, com acesso apenas aos bytes brutos do pacote via estrutura `xdp_md`. As ações possíveis são `XDP_PASS` (entregar à pilha normal), `XDP_DROP` (descartar), `XDP_TX` (retransmitir pela mesma interface) e `XDP_REDIRECT` (encaminhar para outra interface ou socket) (GREGG, 2019).

TC (*Traffic Control*). Executa após a alocação do `sk_buff`, com acesso a metadados de contexto mais ricos do que o XDP — incluindo informações de cgroup, namespace e marcações de conexão (Linux Kernel Documentation, 2024). Opera em dois pontos: `ingress` (entrada, antes do roteamento) e `egress` (saída, após a decisão de roteamento). As ações possíveis incluem `TC_ACT_OK` (continuar), `TC_ACT_SHOT` (descartar) e `TC_ACT_REDIRECT` (redirecionar) (CASSAGNES *et al.*, 2020).

***Tracepoints* de socket.** Pontos de rastreamento estáticos sobre operações de socket — criação, conexão, envio, recepção e encerramento — que fornecem visibilidade sobre o comportamento de rede por processo sem possibilidade de intervenção no fluxo de pacotes (Linux Kernel Documentation, 2024). São o *hook* preferencial para monitoramento de desempenho de rede em ambientes NFV, pois associam métricas de latência e vazão diretamente à função virtualizada responsável (VENÂNCIO *et al.*, 2021).

***Cgroup* de socket (`BPF_PROG_TYPE_CGROUP_SOCK`).** Programas desse tipo são executados quando um processo do cgroup cria ou conecta um socket, permitindo aplicar políticas de rede com escopo por contêiner ou por função virtualizada de rede (Linux Kernel Documentation, 2024). São a base dos controles de rede por identidade implementados por projetos como o Cilium em clusters Kubernetes (CASSAGNES *et al.*, 2020).

3.4 EBPF PROGRAMS

Um programa eBPF voltado para rede segue o mesmo modelo de execução descrito para aplicações de segurança, com variações no tipo de programa, no contexto recebido como argumento e nas *helper functions* disponíveis (Linux Kernel Documentation, 2024).

O tipo do programa determina o *hook* compatível e o contexto passado: programas XDP recebem um ponteiro para `struct xdp_md`, que expõe os limites do buffer do pacote; programas TC recebem um ponteiro para `struct __sk_buff`, uma visão segura do `sk_buff` do kernel com campos de metadados acessíveis; programas de *tracepoint* de socket recebem os argumentos da chamada de sistema correspondente (Linux Kernel Documentation, 2024).

O exemplo a seguir ilustra um programa XDP mínimo que conta pacotes recebidos por protocolo de camada 4 e os armazena em um mapa BPF:

Listing 3.1 – Programa XDP que conta pacotes por protocolo L4.

```
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 256);
    __type(key, __u32);
    __type(value, __u64);
} pkt_count SEC(".maps");

SEC("xdp")
int conta_pacotes(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    __u32 proto = ip->protocol;
    __u64 *cnt  = bpf_map_lookup_elem(&pkt_count, &proto);
    if (cnt)
        __sync_fetch_and_add(cnt, 1);

    return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

As verificações de limites (`data_end`) são obrigatórias: o verificador do kernel rejeita qualquer programa que acesse memória do pacote sem confirmar que o acesso está dentro dos limites do buffer (Linux Kernel Documentation, 2024). Essa propriedade garante que programas XDP não possam causar leituras fora dos limites mesmo em pacotes malformados (GREGG, 2019).

3.5 EBPF MAPS

Mapas BPF são a estrutura de dados central para comunicação entre o programa eBPF no kernel e a aplicação de monitoramento em espaço de usuário (Linux Kernel Documentation, 2024). No contexto de redes, eles acumulam métricas por fluxo, por interface, por processo ou por protocolo, e são lidos periodicamente pela aplicação de monitoramento para exportação a sistemas como Prometheus (Prometheus Authors, s.d.).

Os tipos de mapa mais utilizados em aplicações de monitoramento de rede são:

`BPF_MAP_TYPE_HASH`**.** Tabela de dispersão de propósito geral indexada por chave arbitrária. Em monitoramento de rede, é utilizada para rastrear estados de conexões TCP — com a tupla (IP de origem, IP de destino, porta de origem, porta de destino) como chave — e acumular métricas por fluxo como bytes transferidos, número de pacotes e latência média (GREGG, 2019).

`BPF_MAP_TYPE_PERCPU_HASH` **e** `BPF_MAP_TYPE_PERCPU_ARRAY`**.** Variantes por CPU dos tipos acima, em que cada núcleo do processador mantém sua própria cópia do mapa (Linux Kernel Documentation, 2024). Eliminam a necessidade de operações atômicas em atualizações de contadores — o custo predominante em programas XDP de alta taxa de pacotes — e são consolidados pela aplicação em espaço de usuário ao agregar os valores de todas as CPUs (GREGG, 2020).

`BPF_MAP_TYPE_RINGBUF`**.** Buffer circular de eventos para comunicação assíncrona de alta eficiência entre kernel e espaço de usuário, preferido em relação ao `BPF_MAP_TYPE_PERF_EVENT_ARRAY` a partir do kernel 5.8 (Linux Kernel Documentation, 2024). Em monitoramento de rede, é utilizado para emitir registros de eventos de conexão — abertura, encerramento, retransmissão — com contexto completo para a aplicação de monitoramento, que os encaminha ao `ebpf_exporter` (Cloudflare, s.d.) ou ao Prometheus (Prometheus Authors, s.d.).

`BPF_MAP_TYPE_LPM_TRIE`**.** Árvore de prefixos de comprimento mais longo (*Longest Prefix Match*), projetada para pesquisa eficiente de endereços IP em tabelas de roteamento e listas de controle de acesso de rede (Linux Kernel Documentation, 2024). Permite implementar em um programa XDP ou TC consultas ao estilo "este endereço IP pertence à sub-rede X?" com complexidade logarítmica em relação ao número de prefixos, sem necessidade de iterar sobre uma lista linear (GREGG, 2019).

A cadeia completa de coleta e exportação de métricas utilizada no minicurso combina esses tipos de mapa da seguinte forma: programas XDP ou TC acumulam contadores em mapas `PERCPU_ARRAY`; o `ebpf_exporter` (Cloudflare, s.d.) lê esses mapas em intervalos configuráveis, consolida os valores por CPU e os expõe como métricas Prometheus (Prometheus Authors, s.d.); o Grafana (Grafana Labs, s.d.) consome essas métricas e as exibe em dashboards de monitoramento de funções virtualizadas de rede (ALVES; PLINTA, 2026).

4 XDP — *EXPRESS DATA PATH*

O *eXpress Data Path* (XDP) é o ponto de intervenção mais precoce disponível para programas eBPF na pilha de rede do kernel Linux (Linux Kernel Documentation, 2024). Ao executar diretamente no driver da interface de rede — antes da alocação do `sk_buff` e antes de qualquer processamento pela pilha TCP/IP — o XDP permite processar pacotes com latência mínima e overhead desprezível em comparação com abordagens em espaço de usuário (CASSAGNES *et al.*, 2020). Em ambientes de funções virtualizadas de rede, onde o desempenho de encaminhamento e a capacidade de monitoramento sem intrusão são requisitos fundamentais, o XDP representa uma das contribuições mais relevantes do eBPF (VENÂNCIO *et al.*, 2021).

4.1 MODOS DE EXECUÇÃO

O XDP suporta três modos de execução, que diferem em como o programa eBPF é integrado ao driver da interface de rede e, consequentemente, no desempenho e na compatibilidade obtidos (Linux Kernel Documentation, 2024).

XDP Nativo (*Native XDP*). O programa eBPF é executado diretamente no contexto do driver da interface de rede, antes da alocação do `sk_buff` (Linux Kernel Documentation, 2024). Este modo oferece o melhor desempenho possível, pois elimina toda a sobrecarga de inicialização da pilha de rede para pacotes que serão descartados ou redirecionados. Requer suporte explícito no driver da interface de rede; a maioria dos drivers de alto desempenho — como `mlx5`, `i40e`, `ixgbe` e `virtio_net` — já oferece esse suporte (GREGG, 2019).

XDP Genérico (*Generic XDP*). Disponível em qualquer interface de rede, independentemente do suporte do driver (Linux Kernel Documentation, 2024). Neste modo, o programa eBPF é executado após a alocação do `sk_buff`, dentro do núcleo da pilha de rede, o que significa que o overhead de alocação de memória já foi incorrido antes da execução do programa. O desempenho é inferior ao XDP nativo, mas este modo é útil para desenvolvimento, testes e implantação em interfaces sem suporte nativo, como as de máquinas virtuais mais simples (GREGG, 2020).

XDP Offload (*Offloaded XDP*). O programa eBPF é compilado e carregado diretamente no firmware da placa de rede, que o executa sem qualquer envolvimento da CPU do host (Linux Kernel Documentation, 2024). Este modo oferece o máximo de desempenho e a menor latência possível, pois o processamento ocorre inteiramente no hardware de rede. Atualmente, o suporte é limitado a um pequeno conjunto de placas de rede de alta performance com capacidade de processamento programável, como as da família Netronome Agilio (GREGG, 2019).

A verificação do modo suportado pela interface pode ser feita com:

Listing 4.1 – Verificação do modo XDP suportado pela interface de rede.

```
ip link show eth0
# Saida esperada para XDP nativo ativo:
# xdp/id:42  <BROADCAST,MULTICAST,UP,LOWER_UP>
```

Para fins didáticos, o minicurso utiliza o modo genérico em máquinas virtuais, garantindo compatibilidade independentemente do driver de rede disponível (PLINTA; NAHAS, 2026).

4.2 XDP E PROCESSAMENTO DE PACOTES

Um programa XDP recebe como argumento um ponteiro para a estrutura `struct xdp_md`, que expõe os limites do buffer do pacote em memória (Linux Kernel Documentation, 2024):

Listing 4.2 – Estrutura de contexto de um programa XDP.

```
struct xdp_md {
    __u32 data;          /* inicio dos dados do pacote  */
    __u32 data_end;      /* fim dos dados do pacote     */
    __u32 data_meta;     /* area de metadados           */
    __u32 ingress_ifindex;
    __u32 rx_queue_index;
};
```

O acesso aos bytes do pacote é feito por meio de ponteiros calculados a partir de `data` e `data_end`. Toda operação de leitura ou escrita deve ser precedida de verificação explícita de limites, que o verificador do kernel valida estaticamente (Linux Kernel Documentation, 2024):

Listing 4.3 – Acesso seguro ao cabeçalho Ethernet em um programa XDP.

```
void *data     = (void *)(long)ctx->data;
void *data_end = (void *)(long)ctx->data_end;

struct ethhdr *eth = data;
if ((void *)(eth + 1) > data_end)
    return XDP_PASS;   /* pacote muito curto; entregar a pilha */
```

Ao final do processamento, o programa retorna um código de ação que determina o destino do pacote (Linux Kernel Documentation, 2024):

- `XDP_PASS` — entrega o pacote à pilha de rede normal do kernel para processamento convencional;
- `XDP_DROP` — descarta o pacote imediatamente, sem alocar `sk_buff` nem percorrer a pilha TCP/IP; é a ação de menor custo computacional e a preferida para mitigação de DDoS (GREGG, 2019);

- `XDP_TX` — retransmite o pacote pela mesma interface de entrada, possivelmente após modificação de cabeçalhos;
- `XDP_REDIRECT` — encaminha o pacote para outra interface de rede, para uma CPU diferente ou diretamente para um socket em espaço de usuário via AF_XDP (Linux Kernel Documentation, 2024);
- `XDP_ABORTED` — indica erro de programa; o pacote é descartado e um evento de rastreamento é gerado para depuração.

Para monitoramento de funções virtualizadas de rede, a ação `XDP_PASS` combinada com coleta de métricas em mapas BPF é o padrão mais utilizado: o programa observa cada pacote — contando bytes, registrando protocolos e medindo intervalos entre chegadas — sem interferir no fluxo normal de dados (VENÂNCIO *et al.*, 2021).

O exemplo a seguir demonstra um programa XDP de monitoramento que acumula bytes recebidos por protocolo IP em um mapa `PERCPU_ARRAY` para exportação via Prometheus:

Listing 4.4 – Programa XDP de monitoramento de bytes por protocolo L4.

```
SEC("xdp")
int monitor_bytes(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;

    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    __u32 proto  = ip->protocol;
    __u32 nbytes = (void *)data_end - data;

    __u64 *cnt = bpf_map_lookup_elem(&bytes_by_proto, &proto);
    if (cnt)
        *cnt += nbytes;

    return XDP_PASS;
}
```

4.3 REDIRECIONAMENTO DE PACOTES

O redirecionamento de pacotes com `XDP_REDIRECT` é um dos casos de uso mais poderosos do XDP para funções virtualizadas de rede, pois permite implementar encaminhamento de alta velocidade inteiramente em eBPF, sem passar pela pilha de roteamento do kernel (Linux Kernel Documentation, 2024).

Redirecionamento entre interfaces. A *helper function* `bpf_redirect` encaminha o pacote para uma interface de saída identificada por seu índice (`ifindex`):

Listing 4.5 – Redirecionamento de pacote para outra interface com XDP.

```
SEC("xdp")
int encaminhar(struct xdp_md *ctx)
{
    /* encaminha todos os pacotes para a interface de indice 3 */
    return bpf_redirect(3, 0);
}
```

Esse padrão é utilizado para implementar pontes (*bridges*) e roteadores de alto desempenho em software, substituindo soluções baseadas em *iptables* com ganhos expressivos de throughput (GREGG, 2019).

Redirecionamento para socket AF_XDP. O AF_XDP é uma família de sockets que permite que aplicações em espaço de usuário recebam pacotes diretamente do driver de rede, sem passar pela pilha TCP/IP, por meio de filas de memória compartilhada (*UMEM*) entre kernel e espaço de usuário (Linux Kernel Documentation, 2024). A *helper function* `bpf_redirect_map` com um mapa do tipo `BPF_MAP_TYPE_XSKMAP` é usada para selecionar quais pacotes são entregues a sockets AF_XDP, enquanto os demais seguem o caminho normal com `XDP_PASS`:

Listing 4.6 – Redirecionamento seletivo de pacotes UDP para socket AF_XDP.

```
SEC("xdp")
int xsk_redirect(struct xdp_md *ctx)
{
    /* ... parse dos cabecalhos ... */
    if (ip->protocol == IPPROTO_UDP) {
        __u32 idx = ctx->rx_queue_index;
        return bpf_redirect_map(&xsk_map, idx, XDP_PASS);
    }
    return XDP_PASS;
}
```

Em ambientes NFV, o AF_XDP é utilizado por funções de rede que precisam processar pacotes em espaço de usuário com latência próxima ao kernel — como firewalls e balanceadores de carga implementados em *Data Plane Development Kit* (DPDK) — mas com a vantagem de

interoperabilidade com a pilha de rede padrão do Linux para o tráfego não selecionado (VENÂNCIO *et al.*, 2021).

4.4 VANTAGENS E LIMITAÇÕES

Vantagens.

Desempenho. O XDP nativo é a abordagem de maior desempenho para processamento de pacotes em software no Linux, superando soluções baseadas em *iptables* e em módulos de kernel convencionais em cenários de alta taxa de pacotes (CASSAGNES *et al.*, 2020). A ausência de alocação de `sk_buff` para pacotes descartados ou redirecionados elimina o principal gargalo de memória da pilha de rede em cargas de trabalho de filtragem (GREGG, 2019).

Programabilidade dinâmica. Programas XDP podem ser carregados e substituídos em tempo de execução sem interromper o tráfego de rede, ao contrário de filtros estáticos de *iptables* que exigem reinicialização do serviço de firewall (Linux Kernel Documentation, 2024). Mapas BPF compartilhados entre o programa e o espaço de usuário permitem atualizar configurações — listas de endereços bloqueados, tabelas de roteamento, limiares de monitoramento — sem recarregar o programa (GREGG, 2020).

Visibilidade sem intrusão. A ação `XDP_PASS` permite que programas XDP observem todo o tráfego de uma interface e coletem métricas detalhadas sem alterar o comportamento da rede — propriedade fundamental para monitoramento de funções virtualizadas de rede em produção (VENÂNCIO *et al.*, 2021).

Limitações.

Acesso restrito ao contexto do pacote. No modo nativo, o programa XDP tem acesso apenas aos bytes do pacote via `xdp_md` — sem acesso a metadados de cgroup, namespace ou PID do processo originador, que só estão disponíveis em *hooks* posteriores na pilha, como o TC (Linux Kernel Documentation, 2024). Para monitoramento que exige associação de fluxos de rede a funções virtualizadas específicas, o TC ou *tracepoints* de socket oferecem visibilidade mais rica.

Suporte limitado de drivers. O XDP nativo requer suporte explícito no driver da interface de rede (Linux Kernel Documentation, 2024). Em ambientes de máquina virtual com drivers paravirtualizados menos comuns, pode ser necessário recorrer ao XDP genérico, com desempenho inferior (GREGG, 2020).

Ausência de estado de conexão. O XDP opera pacote a pacote e não mantém estado de conexão nativo (Linux Kernel Documentation, 2024). Casos de uso que exigem rastreamento de fluxos TCP — como detecção de retransmissões ou medição de RTT — precisam manter esse estado explicitamente em mapas BPF, com o cuidado adicional de gerenciar a expiração de entradas para evitar esgotamento de memória (GREGG, 2019).

Restrições do verificador. Como todo programa eBPF, programas XDP estão sujeitos às restrições do verificador — limite de complexidade de laços, verificação obrigatória de limites de acesso a memória e ausência de recursão (Linux Kernel Documentation, 2024). Lógica de processamento muito complexa pode não ser verificável e precisar ser distribuída entre múltiplos programas encadeados via *tail calls* (GREGG, 2019).

5 EBPF E TRAFFIC CONTROL

O subsistema de *Traffic Control* (TC) é responsável pelo enfileiramento, escalonamento, classificação e modelagem do tráfego de rede no kernel Linux (Linux Kernel Documentation, 2024). Ao contrário do XDP, que opera antes da alocação do `sk_buff`, o TC intervém após a camada de enlace ter processado o quadro, dispondo de acesso ao contexto completo do pacote — incluindo metadados de cgroup, namespace e processo originador (CASSAGNES et al., 2020). Essa posição mais tardia na pilha implica maior overhead por pacote em comparação ao XDP nativo, mas oferece uma visibilidade significativamente mais rica, tornando o TC o hook preferencial para implementações de políticas de rede por identidade em ambientes de funções virtualizadas de rede e clusters Kubernetes (GREGG, 2019).

A integração do eBPF ao TC é realizada por meio da *qdisc* `clsact`, introduzida no kernel 4.5, que expõe dois pontos de ancoragem independentes: um em entrada (*ingress*) e outro em saída (*egress*) (Linux Kernel Documentation, 2024). Programas eBPF são anexados a esses pontos como classificadores (`cls_bpf`) e recebem como argumento um ponteiro para `struct __sk_buff` — uma visão segura e anotada do `sk_buff` interno do kernel, que expõe campos como endereços IP de origem e destino, portas, protocolo, tamanho do pacote, interface de entrada e identificador de cgroup do processo responsável pelo fluxo (CASSAGNES et al., 2020).

5.1 TC INGRESS

O ponto de ancoragem *ingress* do TC é avaliado logo após a camada de enlace processar o quadro de entrada e antes que o kernel tome qualquer decisão de roteamento (Linux Kernel Documentation, 2024). Nessa posição, o programa eBPF tem acesso ao `sk_buff` já inicializado — com ponteiros para os cabeçalhos L2, L3 e L4 — e pode inspecionar, modificar, descartar ou redirecionar o pacote antes que ele alcance a pilha IP (GREGG, 2019).

A configuração do ponto de ancoragem *ingress* utiliza a *qdisc* `clsact`, que deve ser anexada à interface antes do carregamento do programa:

Listing 5.1 – Configuração da qdisc clsact e anexação de programa eBPF no ingress.

```
# Adicionar a qdisc clsact a interface eth0
tc qdisc add dev eth0 clsact

# Carregar o objeto BPF e anexar ao ingress
tc filter add dev eth0 ingress bpf da obj tc_monitor.o sec tc_ingress
```

Um programa TC *ingress* recebe `struct __sk_buff` como contexto. O exemplo a seguir demonstra um classificador de entrada que acumula bytes recebidos por protocolo de camada 4 e registra eventos de novas conexões TCP em um mapa `BPF_MAP_TYPE_RINGBUF`

para exportação ao espaço de usuário:

Listing 5.2 – Programa TC ingress que monitora bytes e novas conexões TCP.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

struct conn_event {
    __u32 src_ip;
    __u32 dst_ip;
    __u16 src_port;
    __u16 dst_port;
    __u32 pid;
};

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 256);
    __type(key, __u32);
    __type(value, __u64);
} bytes_in SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 16);
} conn_events SEC(".maps");

SEC("tc_ingress")
int monitor_ingress(struct __sk_buff *skb)
{
    void *data     = (void *)(long)skb->data;
    void *data_end = (void *)(long)skb->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return TC_ACT_OK;

    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return TC_ACT_OK;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return TC_ACT_OK;
```

```
    /* Acumula bytes por protocolo L4 */
    __u32 proto  = ip->protocol;
    __u64 nbytes = data_end - data;
    __u64 *cnt   = bpf_map_lookup_elem(&bytes_in, &proto);
    if (cnt)
        *cnt += nbytes;

    /* Registra novas conexoes TCP (flag SYN) */
    if (proto == IPPROTO_TCP) {
        struct tcphdr *tcp = (void *)(ip + 1);
        if ((void *)(tcp + 1) > data_end)
            return TC_ACT_OK;

        if (tcp->syn && !tcp->ack) {
            struct conn_event *ev =
                bpf_ringbuf_reserve(&conn_events,
                                    sizeof(*ev), 0);
            if (ev) {
                ev->src_ip   = ip->saddr;
                ev->dst_ip   = ip->daddr;
                ev->src_port = bpf_ntohs(tcp->source);
                ev->dst_port = bpf_ntohs(tcp->dest);
                ev->pid      = bpf_get_current_pid_tgid() >> 32;
                bpf_ringbuf_submit(ev, 0);
            }
        }
    }

    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

O código de retorno `TC_ACT_OK` instrui o kernel a continuar o processamento normal do pacote. As demais ações possíveis são `TC_ACT_SHOT` (descartar o pacote), `TC_ACT_REDIRECT` (redirecionar para outra interface) e `TC_ACT_STOLEN` (consumir o pacote sem entregá-lo à pilha) (Linux Kernel Documentation, 2024).

Em ambientes de funções virtualizadas de rede, o *ingress* TC é amplamente utilizado para monitorar o tráfego recebido por cada função virtualizada com granularidade de fluxo, associando cada pacote ao cgroup da função responsável por meio do campo `skb->cgroup_classid` do contexto `__sk_buff` (VENÂNCIO et al., 2021). Essa granularidade é inacessível ao XDP, que opera antes da inicialização dos metadados de cgroup no `sk_buff` (CASSAGNES et al., 2020).

5.2 TC EGRESS

O ponto de ancoragem *egress* do TC é avaliado após a decisão de roteamento e antes que o pacote seja entregue ao driver da interface de saída para transmissão (Linux Kernel Documentation, 2024). Nessa posição, o programa eBPF tem acesso ao contexto completo do `sk_buff` de saída — incluindo o PID e o cgroup do processo que originou o pacote — e pode implementar políticas de saída com escopo por contêiner ou por função virtualizada de rede (CASSAGNES et al., 2020).

A configuração do ponto de ancoragem *egress* é análoga à do *ingress*, utilizando a mesma *qdisc* `clsact` adicionada anteriormente:

Listing 5.3 – Anexação de programa eBPF no egress TC.

```
tc filter add dev eth0 egress bpf da obj tc_monitor.o sec tc_egress
```

O exemplo a seguir demonstra um programa TC *egress* que mede a taxa de transmissão por cgroup e impõe um limite máximo de bytes por segundo descartando pacotes excedentes — implementando uma forma simples de *policing* de saída sem dependência de módulos externos:

Listing 5.4 – Programa TC egress com monitoramento de bytes por cgroup.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <bpf/bpf_helpers.h>

#define RATE_LIMIT_BYTES 1250000ULL  /* ~10 Mbps por cgroup */

struct rate_state {
    __u64 bytes;
    __u64 last_ts;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u64);           /* cgroup id */
    __type(value, struct rate_state);
} egress_rate SEC(".maps");

SEC("tc_egress")
int monitor_egress(struct __sk_buff *skb)
{
    __u64 cgid = bpf_skb_cgroup_id(skb);
    __u64 now  = bpf_ktime_get_ns();
    __u32 pkt_len = skb->len;

    struct rate_state *rs =
```

```
        bpf_map_lookup_elem(&egress_rate, &cgid);

    if (!rs) {
        struct rate_state init = { .bytes = pkt_len,
                                   .last_ts = now };
        bpf_map_update_elem(&egress_rate, &cgid,
                            &init, BPF_ANY);
        return TC_ACT_OK;
    }

    /* Reinicia contador a cada segundo */
    if (now - rs->last_ts > 1000000000ULL) {
        rs->bytes   = 0;
        rs->last_ts = now;
    }

    rs->bytes += pkt_len;

    if (rs->bytes > RATE_LIMIT_BYTES)
        return TC_ACT_SHOT;   /* descarta excedente */

    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

O campo `skb->len` expõe o tamanho total do pacote, incluindo cabeçalhos, sem necessidade de recalculá-lo a partir dos ponteiros de dados (Linux Kernel Documentation, 2024). A *helper function* `bpf_skb_cgroup_id` retorna o identificador do cgroup v2 ao qual pertence o socket originador do pacote, permitindo aplicar políticas distintas a cada função virtualizada de rede ou contêiner (GREGG, 2019).

Uma diferença importante entre *ingress* e *egress* no TC é a disponibilidade de informações de processo. No *egress*, o pacote ainda está associado ao contexto do processo que o gerou, de modo que funções como `bpf_get_current_pid_tgid` e `bpf_get_current_comm` retornam valores válidos para a maioria dos fluxos (GREGG, 2020). No *ingress*, o processo originador é o emissor remoto, inacessível localmente; o cgroup do receptor pode ser obtido por meio de `bpf_skb_cgroup_id`, mas somente após a associação do pacote a um socket pelo kernel (CASSAGNES et al., 2020).

5.3 MANIPULAÇÃO, CLASSIFICAÇÃO E CONTROLE DE TRÁFEGO

Além de coletar métricas, programas TC eBPF podem modificar ativamente os pacotes em trânsito — alterando cabeçalhos, remarcando campos de qualidade de serviço ou reescre-

vendo endereços — e implementar lógica de classificação e controle de tráfego que anteriormente exigia módulos de kernel dedicados ou ferramentas de espaço de usuário com overhead elevado (Linux Kernel Documentation, 2024).

Manipulação de cabeçalhos

A *helper function* `bpf_skb_store_bytes` permite modificar bytes arbitrários do pacote diretamente no `sk_buff`, e `bpf_l3_csum_replace` e `bpf_l4_csum_replace` atualizam os checksums dos cabeçalhos IP e TCP/UDP após qualquer modificação (Linux Kernel Documentation, 2024). O exemplo a seguir demonstra a reescrita do campo DSCP (*Differentiated Services Code Point*) do cabeçalho IP para marcar pacotes de um cgroup específico com alta prioridade:

Listing 5.5 – Remarcação de DSCP para pacotes de cgroup prioritário.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <linux/ip.h>
#include <bpf/bpf_helpers.h>

#define PRIO_CGID  1234ULL   /* cgroup id prioritario */
#define DSCP_EF    0xB8      /* Expedited Forwarding (46 << 2) */

SEC("tc_egress")
int remarcar_dscp(struct __sk_buff *skb)
{
    if (bpf_skb_cgroup_id(skb) != PRIO_CGID)
        return TC_ACT_OK;

    /* Offset do campo TOS no cabecalho IP: 14 bytes Ethernet + 1 */
    __u8 tos = DSCP_EF;
    bpf_skb_store_bytes(skb, ETH_HLEN + offsetof(struct iphdr, tos),
                        &tos, sizeof(tos), BPF_F_RECOMPUTE_CSUM);

    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

Essa técnica é utilizada em ambientes NFV para diferenciar o tráfego de funções virtualizadas de rede com requisitos de latência distintos — como funções de plano de controle e funções de plano de dados — sem modificar as próprias funções (VENÂNCIO et al., 2021).

Classificação de tráfego

Programas TC eBPF podem classificar pacotes em classes de tráfego utilizando o campo `skb->tc_classid`, que é lido pelas *qdiscs* hierárquicas como `HTB` (*Hierarchical Token Bucket*) para aplicar limites de banda diferenciados (Linux Kernel Documentation, 2024). Essa integração permite combinar a flexibilidade de programas eBPF — que podem inspecionar qualquer campo do pacote ou consultar mapas BPF com listas de classificação — com a maturidade dos mecanismos de enfileiramento existentes no TC (GREGG, 2019):

Listing 5.6 – Classificação de pacotes por porta de destino para HTB.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#define TC_H_MAKE(maj, min) (((maj) << 16) | (min))
#define CLASS_HIGH  TC_H_MAKE(1, 10)  /* classe de alta prioridade */
#define CLASS_LOW   TC_H_MAKE(1, 20)  /* classe de baixa prioridade */
#define PORT_HTTP   80
#define PORT_BACKUP 873

SEC("tc_egress")
int classificar_trafego(struct __sk_buff *skb)
{
    void *data     = (void *)(long)skb->data;
    void *data_end = (void *)(long)skb->data_end;

    struct iphdr *ip = data + sizeof(struct ethhdr);
    if ((void *)(ip + 1) > data_end)
        return TC_ACT_OK;

    if (ip->protocol != IPPROTO_TCP)
        return TC_ACT_OK;

    struct tcphdr *tcp = (void *)(ip + 1);
    if ((void *)(tcp + 1) > data_end)
        return TC_ACT_OK;

    __u16 dport = bpf_ntohs(tcp->dest);

    if (dport == PORT_HTTP)
        skb->tc_classid = CLASS_HIGH;
    else if (dport == PORT_BACKUP)
        skb->tc_classid = CLASS_LOW;
```

```
    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

Controle de tráfego e redirecionamento

O redirecionamento de pacotes no TC é realizado pela *helper function* `bpf_redirect`, análoga à usada no XDP, mas disponível com acesso ao contexto completo do `sk_buff` (Linux Kernel Documentation, 2024). Isso permite implementar no *ingress* TC uma forma de balanceamento de carga por *software*: pacotes destinados a um endereço IP virtual são redirecionados para interfaces distintas com base em informações de estado mantidas em mapas BPF, sem necessidade de módulos de kernel adicionais (CASSAGNES et al., 2020).

Em comparação com o XDP, o TC apresenta trade-offs claros: maior overhead por pacote devido à posição mais tardia na pilha, porém visibilidade de metadados de processo e cgroup inacessíveis ao XDP, suporte nativo em qualquer interface de rede sem dependência do driver e compatibilidade direta com as *qdiscs* hierárquicas existentes (GREGG, 2019). Para aplicações de monitoramento de funções virtualizadas de rede — em que associar métricas de rede a funções específicas é um requisito central — o TC representa o hook de escolha, complementando o XDP em cenários de alto desempenho onde a granularidade por fluxo é igualmente necessária (VENÂNCIO et al., 2021).

6 OBSERVABILIDADE DE REDE

A observabilidade de rede com eBPF consiste em instrumentar a pilha de comunicação do kernel para coletar métricas de desempenho, eventos de conexão e fluxos de tráfego com granularidade por processo, por cgroup e por interface — sem modificar as aplicações monitoradas e sem gerar tráfego adicional de sonda (GREGG, 2019). Em ambientes de funções virtualizadas de rede, essa capacidade é especialmente relevante: múltiplas funções compartilham o mesmo host físico, e distinguir o comportamento de rede de cada uma delas exige associar métricas ao contexto de execução de cada função (VENÂNCIO et al., 2021).

A cadeia de observabilidade adotada no minicurso combina programas eBPF no kernel — anexados a hooks XDP, TC ou tracepoints de socket — com o `ebpf_exporter` (Cloudflare, s.d.) para leitura periódica dos mapas BPF e exposição das métricas no formato Prometheus (Prometheus Authors, s.d.), visualizadas em dashboards Grafana (Grafana Labs, s.d.) (ALVES; PLINTA, 2026).

6.1 MONITORAMENTO DE INTERFACES, PACOTES E CONEXÕES

O monitoramento de interfaces, pacotes e conexões com eBPF abrange três dimensões complementares: contadores agregados por interface, inspeção por pacote e rastreamento de eventos de ciclo de vida de conexões (Linux Kernel Documentation, 2024).

Contadores por interface

Contadores de bytes e pacotes por interface de rede podem ser acumulados em mapas `BPF_MAP_TYPE_PERCPU_ARRAY` por programas XDP ou TC *ingress*, eliminando o custo de leitura periódica do sistema de arquivos `/proc/net/dev` e oferecendo resolução temporal superior à do intervalo de coleta do `sysstat` (GODARD; Sysstat Contributors, 2024). O exemplo a seguir demonstra um programa XDP que acumula contadores de pacotes e bytes por interface de entrada:

Listing 6.1 – Contadores de pacotes e bytes por interface com XDP.

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

struct iface_stats {
    __u64 packets;
    __u64 bytes;
};
```

```
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, struct iface_stats);
} iface_counters SEC(".maps");

SEC("xdp")
int contar_pacotes(struct xdp_md *ctx)
{
    __u32 key = 0;
    struct iface_stats *s =
        bpf_map_lookup_elem(&iface_counters, &key);
    if (s) {
        s->packets++;
        s->bytes += ctx->data_end - ctx->data;
    }
    return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

A leitura dos valores por CPU e sua consolidação são realizadas pela aplicação em espaço de usuário, somando as entradas de todas as CPUs antes de exportar a métrica ao Prometheus (GREGG, 2020).

Inspeção por pacote

A inspeção por pacote no nível do TC *ingress* oferece granularidade de protocolo, porta e endereço IP, permitindo construir histogramas de distribuição de tamanho de pacote ou contadores por par de endereços sem gerar tráfego de captura adicional (CASSAGNES et al., 2020). O `bpftrace` permite realizar essa inspeção interativamente sem escrever um programa eBPF completo:

Listing 6.2 – Histograma de tamanho de pacotes TCP com bpftrace.

```
sudo bpftrace -e '
tracepoint:net:netif_receive_skb
{
    @tamanho = hist(args->len);
}
interval:s:5 { print(@tamanho); clear(@tamanho); }
'
```

Rastreamento de eventos de conexão

O rastreamento do ciclo de vida de conexões TCP — abertura, encerramento e retransmissão — é realizado por meio de tracepoints de socket e kprobes sobre funções internas do kernel como `tcp_connect` e `tcp_close` (Linux Kernel Documentation, 2024). A ferramenta `tcpconnect-bpfcc` do BCC (IO Visor Project, s.d.) ilustra esse padrão, exibindo em tempo real cada nova conexão TCP com PID, comando, endereço e porta:

Listing 6.3 – Monitoramento de novas conexões TCP com tcpconnect-bpfcc.

```
sudo tcpconnect-bpfcc -t
```

Para monitoramento de encerramento de conexões e coleta de estatísticas acumuladas por fluxo — como bytes transferidos e número de retransmissões — o padrão de implementação utiliza um mapa `BPF_MAP_TYPE_HASH` indexado pela tupla de quatro elementos do fluxo (endereço de origem, endereço de destino, porta de origem, porta de destino), atualizado em tracepoints de envio e recepção e lido no encerramento da conexão (GREGG, 2019).

6.2 LATÊNCIA

A medição de latência de rede com eBPF é realizada de forma passiva — sem geração de tráfego de sonda — por meio da instrumentação de tracepoints e kprobes no caminho de envio e recepção de pacotes no kernel (RESEARCH, 2024). As principais métricas de latência acessíveis são o RTT (*round-trip time*) estimado pelo TCP e a latência de enfileiramento no subsistema de TC.

RTT passivo por conexão TCP

O kernel Linux mantém uma estimativa de RTT por socket TCP, atualizada a cada confirmação recebida (*ACK*). Essa estimativa é acessível via o campo `srtt_us` da estrutura `tcp_sock`, que pode ser lido por programas eBPF anexados ao tracepoint `tcp:tcp_probe` ou à kprobe `tcp_rcv_established` (Linux Kernel Documentation, 2024). O exemplo a seguir utiliza o `bpftrace` para construir um histograma de RTT por endereço de destino:

Listing 6.4 – Histograma de RTT TCP passivo com bpftrace.

```
sudo bpftrace -e '
tracepoint:tcp:tcp_probe
{
    /* srtt_us esta em microssegundos, dividido por 8 (media movel) */
    @rtt_us[args->daddr] = hist(args->srtt >> 3);
```

```
}
interval:s:10 { print(@rtt_us); clear(@rtt_us); }
'
```

Em ambientes NFV, a medição passiva de RTT por cgroup permite identificar degradação de latência introduzida por funções virtualizadas de rede sem instrumentar as próprias funções (VENÂNCIO et al., 2021). O programa de monitoramento associa o identificador de cgroup ao socket no momento da criação da conexão — via o hook `BPF_PROG_TYPE_CGROUP_SOCK` — e propaga esse identificador como chave do mapa de histogramas de RTT (CASSAGNES et al., 2020).

Latência de enfileiramento no TC

A latência introduzida pelo subsistema de TC — tempo entre a chegada do pacote à *qdisc* e sua entrega ao driver — pode ser medida registrando o timestamp de entrada com `bpf_ktime_get_ns` em um programa TC *ingress* e calculando o delta em um programa TC *egress* ou em um tracepoint de transmissão (Linux Kernel Documentation, 2024). Essa métrica é particularmente relevante em ambientes com *policing* ativo, onde o enfileiramento pode introduzir latência adicional não prevista pelas funções de rede (GREGG, 2019).

Listing 6.5 – Medição de latência de enfileiramento com timestamp BPF.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 65536);
    __type(key, __u32);    /* hash do fluxo */
    __type(value, __u64);  /* timestamp de entrada */
} ts_ingress SEC(".maps");

SEC("tc_ingress")
int marca_timestamp(struct __sk_buff *skb)
{
    __u32 key = skb->hash;
    __u64 ts  = bpf_ktime_get_ns();
    bpf_map_update_elem(&ts_ingress, &key, &ts, BPF_ANY);
    return TC_ACT_OK;
}

SEC("tc_egress")
int mede_latencia(struct __sk_buff *skb)
```

```
{
    __u32 key = skb->hash;
    __u64 *ts = bpf_map_lookup_elem(&ts_ingress, &key);
    if (ts) {
        __u64 delta = bpf_ktime_get_ns() - *ts;
        bpf_printk("latencia_fila_ns=%llu", delta);
        bpf_map_delete_elem(&ts_ingress, &key);
    }
    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

Em produção, a saída de `bpf_printk` é substituída por acumulação em um mapa de histograma `BPF_MAP_TYPE_PERCPU_ARRAY`, lido periodicamente pelo `ebpf_exporter` para exportação ao Prometheus (Cloudflare, s.d.).

6.3 THROUGHPUT

O monitoramento de *throughput* com eBPF consiste em acumular bytes transmitidos e recebidos por unidade de tempo, com granularidade configurável — por interface, por fluxo, por processo ou por cgroup — sem dependência de ferramentas de captura de pacotes em espaço de usuário (GREGG, 2020).

Throughput por interface

O *throughput* agregado por interface é obtido acumulando o campo `skb->len` em mapas `BPF_MAP_TYPE_PERCPU_ARRAY` em programas XDP ou TC e calculando a taxa de transferência na aplicação em espaço de usuário como a variação do contador entre leituras consecutivas (Linux Kernel Documentation, 2024). O `bpftrace` permite verificar rapidamente o *throughput* de entrada de uma interface sem carregar um programa eBPF dedicado:

Listing 6.6 – Throughput de entrada por interface com bpftrace.

```
sudo bpftrace -e '
tracepoint:net:netif_receive_skb
/str(args->name) == "eth0"/
{
    @bytes_in = sum(args->len);
}
interval:s:1
{
```

```
    printf("throughput_in: %llu bytes/s\n", @bytes_in);
    clear(@bytes_in);
}
'
```

Throughput por fluxo e por cgroup

Para monitoramento de *throughput* com granularidade por fluxo, o mapa `BPF_MAP_TYPE_HASH` indexado pela tupla de quatro elementos do fluxo é atualizado em cada pacote com a quantidade de bytes carregados (GREGG, 2019). Em ambientes NFV, a mesma métrica é indexada pelo identificador de cgroup da função virtualizada, obtido via `bpf_skb_cgroup_id` no TC *egress*, permitindo construir séries temporais de *throughput* por função sem instrumentar as próprias funções (VENÂNCIO et al., 2021):

Listing 6.7 – Throughput por cgroup no TC egress.

```
#include <linux/bpf.h>
#include <linux/pkt_cls.h>
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u64);    /* cgroup id */
    __type(value, __u64);  /* bytes acumulados */
} bytes_por_cgroup SEC(".maps");

SEC("tc_egress")
int throughput_egress(struct __sk_buff *skb)
{
    __u64 cgid    = bpf_skb_cgroup_id(skb);
    __u64 nbytes  = skb->len;
    __u64 *cnt    = bpf_map_lookup_elem(&bytes_por_cgroup, &cgid);

    if (cnt) {
        __sync_fetch_and_add(cnt, nbytes);
    } else {
        bpf_map_update_elem(&bytes_por_cgroup, &cgid,
                            &nbytes, BPF_ANY);
    }

    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

A taxa de transferência é calculada na aplicação em espaço de usuário — ou pelo `ebpf_exporter` — como a diferença entre leituras consecutivas do mapa, dividida pelo intervalo de coleta (Cloudflare, s.d.).

6.4 NETWORK FLOWS

Um *network flow* é definido como um conjunto de pacotes que compartilham os mesmos endereços IP de origem e destino, portas e protocolo de transporte durante um intervalo de tempo — a tupla de cinco elementos que identifica unicamente uma conexão na pilha TCP/IP (GREGG, 2019). O rastreamento de fluxos com eBPF permite construir um inventário completo das comunicações de rede de um host, associando cada fluxo ao processo e ao cgroup responsável, sem depender de ferramentas de captura em espaço de usuário (CASSAGNES et al., 2020).

Tabela de fluxos ativos

A tabela de fluxos ativos é mantida em um mapa `BPF_MAP_TYPE_HASH` cujas chaves são structs representando a tupla do fluxo e cujos valores acumulam métricas por fluxo — bytes, pacotes, timestamps de início e de última atividade (Linux Kernel Documentation, 2024). Programas TC *ingress* e *egress* inserem ou atualizam entradas nessa tabela a cada pacote; tracepoints de `tcp_close` e `udp_destroy_sock` removem a entrada ao encerrar o fluxo e emitem o registro final via `BPF_MAP_TYPE_RINGBUF`:

Listing 6.8 – Estruturas de chave e valor para tabela de fluxos ativos.

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

struct flow_key {
    __u32 src_ip;
    __u32 dst_ip;
    __u16 src_port;
    __u16 dst_port;
    __u8  protocol;
    __u8  pad[3];
};

struct flow_stats {
    __u64 bytes;
    __u64 packets;
    __u64 ts_start;   /* bpf_ktime_get_ns no primeiro pacote */
    __u64 ts_last;    /* bpf_ktime_get_ns no ultimo pacote   */
    __u64 cgid;       /* cgroup id do processo originador    */
```

```
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 65536);
    __type(key, struct flow_key);
    __type(value, struct flow_stats);
} flow_table SEC(".maps");
```

Exportação de fluxos encerrados

Ao encerrar um fluxo TCP, a entrada correspondente na tabela é lida, copiada para um evento no `BPF_MAP_TYPE_RINGBUF` e removida do mapa, liberando espaço para novos fluxos (Linux Kernel Documentation, 2024). A aplicação em espaço de usuário consome esses eventos e os encaminha a um coletor compatível com o formato IPFIX ou ao Prometheus para análise histórica (GREGG, 2020):

Listing 6.9 – Emissão de registro de fluxo encerrado via ringbuf.

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

/* Reaproveitando flow_key e flow_stats do Listing 6.8 */

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 20);
} flow_events SEC(".maps");

SEC("tracepoint/tcp/tcp_destroy_sock")
int registrar_fluxo_encerrado(struct trace_event_raw_tcp_event_sk *ctx)
{
    struct flow_key key = {};
    key.src_ip   = ctx->saddr;
    key.dst_ip   = ctx->daddr;
    key.src_port = ctx->sport;
    key.dst_port = ctx->dport;
    key.protocol = IPPROTO_TCP;

    struct flow_stats *fs =
        bpf_map_lookup_elem(&flow_table, &key);
    if (!fs)
        return 0;

    struct flow_stats *ev =
```

```
        bpf_ringbuf_reserve(&flow_events, sizeof(*ev), 0);
    if (ev) {
        *ev = *fs;
        bpf_ringbuf_submit(ev, 0);
    }

    bpf_map_delete_elem(&flow_table, &key);
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

Visualização com bpftrace

Para investigação interativa de fluxos ativos sem carregar um programa eBPF dedicado, o `bpftrace` oferece acesso direto aos tracepoints de socket (bpftrace Contributors, s.d.):

Listing 6.10 – Listagem de novas conexões TCP com origem e destino via bpftrace.

```
sudo bpftrace -e '
tracepoint:tcp:tcp_connect
{
    printf("%-6d %-16s %s:%d -> %s:%d\n",
           pid, comm,
           ntop(args->saddr), args->sport,
           ntop(args->daddr), args->dport);
}
'
```

Em ambientes NFV, a tabela de fluxos eBPF substitui com vantagens o monitoramento baseado em `/proc/net/tcp` — que exige leitura e análise de um arquivo de texto por intervalo de coleta — fornecendo eventos de fluxo em tempo real com associação direta ao cgroup da função virtualizada responsável e sem custo adicional de análise sintática (VENÂNCIO et al., 2021).

7 SEGURANÇA DE REDE

Os capítulos 4 e 5 apresentaram XDP e TC como pontos de intervenção no caminho dos pacotes, e o capítulo 6 mostrou como observar o tráfego sem alterá-lo. Este capítulo usa os mesmos ganchos para o lado defensivo: descartar tráfego indesejado, montar um *firewall* cujas regras vivem em mapas BPF e detectar comportamento suspeito, como uma inundação de pacotes SYN (Linux Kernel Documentation, 2024; GREGG, 2019). Todos os exemplos são didáticos e voltados a ambiente de laboratório; um sistema em produção exige testes de carga, tratamento de IPv6 e política de falha mais cuidadosos do que os apresentados aqui.

Os programas C deste capítulo foram compilados com `clang -target bpf -O2 -g -c`, o que confirma sintaxe e uso correto dos cabeçalhos. A aceitação pelo verificador e o comportamento em tempo de execução dependem de carregar o programa no kernel, o que exige *root*. O filtro XDP `fw_xdp` foi carregado e testado pelo script `testes/09-redes-exemplos.sh` (capítulo 11): o `ping` passou com a lista de bloqueio vazia, foi bloqueado ao inserir a origem e voltou ao remover a regra. O firewall de saída em TC foi carregado no `veth-pod` pelo script `testes/10-pendencias.sh`: o verificador o aceitou, ele bloqueou o `curl` para a porta 8099 (o contador da regra chegou a 5) e o tráfego voltou ao remover a regra. O limitador de SYN teve apenas o carregamento aceito pelo verificador, sem tráfego de teste, e o bloco de contagem de *sockets* em `SYN_RECV` não foi executado.

7.1 FILTRAGEM DE PACOTES

Filtrar pacotes é decidir, para cada um, entre deixá-lo seguir ou descartá-lo com base em campos dos cabeçalhos. Em eBPF, a decisão acontece em um dos dois ganchos já estudados (Linux Kernel Documentation, 2024):

- **XDP** (`XDP_DROP`): o descarte ocorre antes da alocação do `sk_buff`, com o menor custo possível; é o ponto ideal para tráfego de entrada que nunca deve alcançar a pilha, como origens conhecidamente maliciosas (capítulo 4);
- **TC** (`TC_ACT_SHOT`): o descarte ocorre com o `sk_buff` já criado, mas com acesso a metadados como o *cgroup* do originador; é o ponto natural para o tráfego de **saída** e para políticas por contêiner (capítulo 5).

A diferença para o *iptables* e o *nftables* está no modelo de configuração. Regras de *netfilter* são percorridas em cadeias, em sequência; a quantidade de regras influencia o custo por pacote. Em eBPF, a regra é código, e listas grandes de endereços ou portas ficam em mapas `HASH`, com custo de consulta aproximadamente constante (GREGG, 2019). Atualizar a lista é uma escrita no mapa, sem recarregar o programa.

O critério de decisão mais simples é a **lista de bloqueio por endereço de origem**. O programa a seguir consulta o endereço IPv4 de origem do pacote em um mapa e também verifica se a porta de destino TCP ou UDP está proibida. Os dois mapas são preenchidos pelo espaço de usuário; um terceiro mapa, `PERCPU_ARRAY`, conta pacotes aceitos e descartados.

Listing 7.1 – Filtro XDP por IP de origem e porta de destino (fw_xdp.bpf.c).

```
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/udp.h>
#include <linux/in.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u32);            /* IPv4 de origem (ordem de rede) */
    __type(value, __u32);
} blocklist SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 64);
    __type(key, __u16);            /* porta de destino (ordem do host)
   ↪ */
    __type(value, __u32);
} blocked_ports SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 2);        /* 0 = aceitos, 1 = descartados */
    __type(key, __u32);
    __type(value, __u64);
} stats SEC(".maps");

static __always_inline void conta(__u32 idx)
{
    __u64 *v = bpf_map_lookup_elem(&stats, &idx);
    if (v)
        *v += 1;
}

SEC("xdp")
int xdp_fw(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
```

```
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    /* 1. origem na lista de bloqueio */
    __u32 src = ip->saddr;
    if (bpf_map_lookup_elem(&blocklist, &src))
        goto drop;

    /* 2. porta de destino bloqueada (TCP ou UDP) */
    if (ip->protocol == IPPROTO_TCP || ip->protocol == IPPROTO_UDP) {
        void *l4 = (void *)ip + ip->ihl * 4;
        struct udphdr *udp = l4;   /* dest fica no mesmo offset em TCP e
   ↪  UDP */
        if ((void *)(udp + 1) > data_end)
            goto ok;
        __u16 dport = bpf_ntohs(udp->dest);
        if (bpf_map_lookup_elem(&blocked_ports, &dport))
            goto drop;
    }

ok:
    conta(0);
    return XDP_PASS;
drop:
    conta(1);
    return XDP_DROP;
}

char LICENSE[] SEC("license") = "GPL";
```

Três detalhes do programa merecem atenção. Primeiro, o endereço de origem é usado como chave em **ordem de rede**, exatamente como está no pacote, sem conversão; quem preenche o mapa precisa respeitar essa ordem. Segundo, o tamanho do cabeçalho IP é variável (`ihl * 4`), e o programa o respeita antes de ler a porta. Terceiro, em caso de pacote truncado ou protocolo não tratado, a escolha é `XDP_PASS`: o filtro *falha aberto*. Para um filtro de segurança que deve falhar fechado, o retorno nesses ramos seria `XDP_DROP`, ao custo de derrubar tráfego legítimo que o programa não sabe interpretar — uma decisão de política, não técnica.

Compilar o objeto não exige privilégio:

Listing 7.2 – Compilação do programa XDP.

```
clang -target bpf -O2 -g -Wall -I/usr/include/x86_64-linux-gnu \
      -c fw_xdp.bpf.c -o fw_xdp.bpf.o
```

O diretório `-I/usr/include/x86_64-linux-gnu` é necessário em distribuições Debian e Ubuntu para que `<asm/types.h>` seja encontrado ao compilar com `-target bpf`; em outras arquiteturas, o nome do diretório muda.

7.2 FIREWALL BASEADO EM EBPF

Um *firewall* é a combinação de um programa que aplica regras com um mecanismo para administrá-las. No caso do Listing 7.1, as regras são as entradas dos mapas `blocklist` e `blocked_ports`, e o administrador é a própria ferramenta `bpftool` (Linux Kernel Documentation, 2024).

Carga e anexação

O `bpftool prog loadall` carrega o objeto e fixa (*pin*) programas e mapas no sistema de arquivos BPF; o `bpftool net attach` liga o programa à interface. O modo `xdpgeneric` funciona em qualquer interface, inclusive `veth` e máquinas virtuais (seção 4.1):

Listing 7.3 – Carga e anexação do firewall XDP (requer root).

```
sudo mkdir -p /sys/fs/bpf/fw
sudo bpftool prog loadall fw_xdp.bpf.o /sys/fs/bpf/fw \
     pinmaps /sys/fs/bpf/fw/maps
sudo bpftool net attach xdpgeneric pinned /sys/fs/bpf/fw/xdp_fw dev eth0
bpftool net show dev eth0
```

A saída de `bpftool net show` deve listar o programa na seção `xdp`; o formato exato varia com a versão da ferramenta, por isso não é reproduzido aqui.

Administração das regras

Com os mapas fixados, a lista de bloqueio é editada em tempo de execução. A chave é o endereço em bytes, na ordem em que aparece no pacote: `10.0.0.2` corresponde a `0a 00 00 02`. O valor é um inteiro de 32 bits qualquer (apenas a presença da chave importa), escrito em *little-endian* no x86:

Listing 7.4 – Edição das regras do firewall por meio dos mapas.

```
# bloquear a origem 10.0.0.2
sudo bpftool map update pinned /sys/fs/bpf/fw/maps/blocklist \
     key hex 0a 00 00 02 value hex 01 00 00 00

# bloquear a porta de destino 8080 (0x1f90, u16 no formato do host)
sudo bpftool map update pinned /sys/fs/bpf/fw/maps/blocked_ports \
     key hex 90 1f value hex 01 00 00 00

# listar e remover regras
sudo bpftool map dump pinned /sys/fs/bpf/fw/maps/blocklist
sudo bpftool map delete pinned /sys/fs/bpf/fw/maps/blocklist \
     key hex 0a 00 00 02

# contadores de aceitos (indice 0) e descartados (indice 1), por CPU
sudo bpftool map dump pinned /sys/fs/bpf/fw/maps/stats
```

A chave da porta é um `__u16` no formato do *host*: a porta 8080 (`0x1f90`) aparece como `90 1f` em máquinas *little-endian*. Como o mapa `stats` é `PERCPU_ARRAY`, o `map dump` mostra um valor por CPU; o total é a soma deles.

Remoção

Para desfazer o experimento, desanexe o programa da interface e remova os arquivos fixados. Sem a remoção do *pin*, o programa e os mapas permanecem na memória do kernel mesmo depois que nenhum processo os referencia.

Listing 7.5 – Desanexação e limpeza.

```
sudo bpftool net detach xdpgeneric dev eth0
sudo rm -r /sys/fs/bpf/fw
```

Firewall de saída com TC

O XDP só vê tráfego de entrada. Para controlar o que uma máquina ou um contêiner *envia*, usa-se o gancho *egress* do TC (capítulo 5). O programa abaixo descarta segmentos TCP cujo destino seja uma porta proibida, como *telnet* (23), e conta quantos pacotes cada regra descartou. Esse é o padrão de uma política de saída restritiva, útil contra *malware* que tenta abrir conexões para portas incomuns.

Listing 7.6 – Firewall de saída em TC (tc_egress_fw.bpf.c).

```
#include <linux/bpf.h>
```

```
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/in.h>
#include <linux/pkt_cls.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 64);
    __type(key, __u16);            /* porta TCP de destino proibida na
   ↪ saida */
    __type(value, __u64);          /* pacotes descartados */
} saida_proibida SEC(".maps");

SEC("tc")
int tc_egress_fw(struct __sk_buff *skb)
{
    void *data     = (void *)(long)skb->data;
    void *data_end = (void *)(long)skb->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end || eth->h_proto != bpf_htons(
   ↪ ETH_P_IP))
        return TC_ACT_OK;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end || ip->protocol != IPPROTO_TCP)
        return TC_ACT_OK;

    struct tcphdr *tcp = (void *)ip + ip->ihl * 4;
    if ((void *)(tcp + 1) > data_end)
        return TC_ACT_OK;

    __u16 dport = bpf_ntohs(tcp->dest);
    __u64 *n = bpf_map_lookup_elem(&saida_proibida, &dport);
    if (n) {
        __sync_fetch_and_add(n, 1);
        return TC_ACT_SHOT;
    }
    return TC_ACT_OK;
}

char LICENSE[] SEC("license") = "GPL";
```

A anexação usa a *qdisc* `clsact`, como no Listing 5.3. A seção `SEC("tc")` é aceita pelo

`tc` a partir de versões recentes do iproute2; em versões antigas, informe a seção explicitamente com `sec tc`.

Listing 7.7 – Anexação do firewall de saída e inclusão de uma regra.

```
sudo tc qdisc add dev eth0 clsact
sudo tc filter add dev eth0 egress bpf da obj tc_egress_fw.bpf.o sec tc
tc filter show dev eth0 egress

# o mapa nao esta fixado: descubra o id e atualize a regra (porta 23)
sudo bpftool map show name saida_proibida
sudo bpftool map update name saida_proibida \
     key hex 17 00 value hex 00 00 00 00 00 00 00 00
```

Limitações do modelo. Os dois programas tratam apenas IPv4 sem opções de encapsulamento. Um firewall completo também precisa de IPv6, de fragmentos IP (cuja porta só aparece no primeiro fragmento) e de *rastreamento de conexão*, para aceitar respostas a conexões iniciadas de dentro e bloquear o restante. O XDP não mantém esse estado sozinho; implementá-lo em mapas é possível, mas é justamente o trabalho que o *conntrack* do *netfilter* já faz. Por isso, soluções maduras como o Cilium (capítulo 9) combinam eBPF com o *conntrack* do kernel ou com tabelas próprias de conexões (Cilium Authors, s.d.a).

7.3 DETECÇÃO DE TRÁFEGO SUSPEITO

Descartar o que já se sabe ser ruim é metade do problema; a outra metade é descobrir o que é ruim. Uma assinatura simples e muito comum é a **inundação de SYN** (*SYN flood*): uma origem envia uma rajada de segmentos SYN para esgotar a fila de conexões semiabertas do servidor. Detectá-la requer contar SYNs iniciais (flag `SYN` ligada e `ACK` desligada) por endereço de origem em uma janela de tempo (GREGG, 2019).

O programa abaixo mantém, para cada origem, o início da janela atual e o número de SYNs vistos nela. Passado o limite, os SYNs excedentes são descartados e contados. O mapa é do tipo `LRU_HASH`: quando cheio, o kernel descarta as entradas menos recentes, o que impede que um atacante esgote a memória do próprio mecanismo de defesa com milhões de endereços forjados — uma preocupação concreta, já que o XDP não tem coleta automática de entradas antigas em um `HASH` comum (capítulo 4).

Listing 7.8 – Limitação de SYN por origem em XDP (syn_guard.bpf.c).

```
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/in.h>
#include <bpf/bpf_helpers.h>
```

```
#include <bpf/bpf_endian.h>

#define JANELA_NS  1000000000ULL   /* janela de 1 s */
#define LIMITE_SYN 100             /* SYNs por origem por janela */

struct syn_info {
    __u64 inicio;   /* inicio da janela atual (ns) */
    __u64 syns;     /* SYNs vistos na janela */
};

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, 65536);
    __type(key, __u32);            /* IPv4 de origem */
    __type(value, struct syn_info);
} syn_por_origem SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);          /* total de SYNs descartados */
} descartados SEC(".maps");

SEC("xdp")
int syn_guard(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end || eth->h_proto != bpf_htons(
   ↪ ETH_P_IP))
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end || ip->protocol != IPPROTO_TCP)
        return XDP_PASS;

    struct tcphdr *tcp = (void *)ip + ip->ihl * 4;
    if ((void *)(tcp + 1) > data_end)
        return XDP_PASS;

    if (!(tcp->syn && !tcp->ack))      /* so SYN inicial */
        return XDP_PASS;

    __u32 src = ip->saddr;
    __u64 now = bpf_ktime_get_ns();
```

```
    struct syn_info *si = bpf_map_lookup_elem(&syn_por_origem, &src);

    if (!si) {
        struct syn_info novo = { .inicio = now, .syns = 1 };
        bpf_map_update_elem(&syn_por_origem, &src, &novo, BPF_ANY);
        return XDP_PASS;
    }
    if (now - si->inicio > JANELA_NS) {
        si->inicio = now;
        si->syns = 1;
        return XDP_PASS;
    }
    /* contagem nao atomica: aproximada com varias CPUs (suficiente aqui
   ↪ ) */
    si->syns++;
    if (si->syns > LIMITE_SYN) {
        __u32 k = 0;
        __u64 *d = bpf_map_lookup_elem(&descartados, &k);
        if (d)
            __sync_fetch_and_add(d, 1);
        return XDP_DROP;
    }
    return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

O incremento `si->syns++` não é atômico: em um sistema com várias CPUs recebendo da mesma origem, algumas contagens podem se perder. Para um limiar de detecção isso é aceitável, pois o erro só atrasa levemente o disparo do limite; para cobrança ou contabilidade exata, use `__sync_fetch_and_add` ou um mapa `PERCPU`.

Escolha do limite. O valor de 100 SYNs por segundo por origem é arbitrário e serve para demonstração. Em produção, ele deve vir da observação do tráfego normal (capítulo 6): um serviço atrás de um *proxy* NAT, por exemplo, concentra muitos clientes legítimos em um único endereço de origem e seria bloqueado por engano. O limite também pode ficar em um mapa de configuração, em vez de uma constante compilada, para ajuste sem recarga.

Observando a detecção

A detecção é útil mesmo sem o descarte. Trocando o `return XDP_DROP` por `XDP_PASS`, o programa vira um detector passivo, e o contador `descartados` passa a indicar quantos SYNs *teriam* sido bloqueados. Essa abordagem, de primeiro observar e depois impor, é a recomendada para qualquer política nova. Para uma verificação rápida e independente do programa, o *bpftrace*

conta SYNs recebidos pelo *tracepoint* de estado de socket (capítulo 6); a contagem por origem aparece ao interromper com `Ctrl-C`:

Listing 7.9 – Contagem de sockets em SYN_RECV por porta local com bpftrace.

```
sudo bpftrace -e 'tracepoint:sock:inet_sock_set_state /args->newstate ==
   ↪  3/ { @syn_recv[args->sport] = count(); }'
```

O estado 3 é `TCP_SYN_RECV`: o *tracepoint* conta os SYNs que chegaram a criar um *socket* em *handshake* na porta local. Um pico indica pressão sobre o serviço; mas SYNs descartados antes disso (por exemplo, pelo XDP) ou que o kernel responde com *syncookies* não aparecem. Para observar o tráfego antes da pilha, o ponto correto é o XDP ou o TC, como no Listing 7.8.

Duas outras assinaturas seguem o mesmo molde de "contar por chave em uma janela": **varredura de portas** (muitas portas de destino distintas para o mesmo destino em pouco tempo, que exige uma chave composta origem–porta) e **inundação UDP** (taxa de pacotes por origem acima do esperado). Ferramentas de segurança em tempo de execução como o Falco (The Falco Authors, s.d.) e o Tetragon (Isovalent, s.d.) aplicam o mesmo princípio a chamadas de sistema; no plano da rede, o Cilium e o Hubble (capítulo 9) fazem o equivalente com identidade de carga de trabalho.

8 EBPF EM CONTAINERS E KUBERNETES

Contêineres são processos comuns do Linux isolados por *namespaces* e *cgroups*; Kubernetes é o orquestrador que decide onde esses contêineres executam e como se comunicam. Esse arranjo cria exigências novas para a rede: milhares de endereços efêmeros, políticas baseadas em rótulos e não em endereços fixos, e balanceamento entre réplicas de um mesmo serviço. O eBPF entra exatamente nesses pontos, pois permite implementar encaminhamento, balanceamento e filtragem dentro do kernel, sem a cadeia de regras do *netfilter* (Linux Kernel Documentation, 2024; Cilium Authors, s.d.a).

Aviso de escopo. Este capítulo é, em sua maior parte, **conceitual**. Não havia um cluster Kubernetes disponível durante a escrita do manual. Os blocos marcados como **executáveis** usam apenas ferramentas do próprio Linux (`ip`, `bpftool`) e rodam em qualquer máquina com privilégio de administrador. Os blocos marcados como **exemplo de manifesto** ou **comando de cluster** descrevem o que se esperaria de um cluster real, mas **não foram executados**; seus manifestos YAML foram apenas verificados quanto à sintaxe.

8.1 NETWORKING EM CONTAINERS E KUBERNETES

O que é um contêiner do ponto de vista da rede

Um contêiner recebe seu próprio *network namespace*: uma pilha de rede isolada, com interfaces, tabela de roteamento e regras de *firewall* próprias. Para conectar esse *namespace* ao restante do sistema, o ambiente cria um par de interfaces virtuais `veth`, que funcionam como as duas pontas de um cabo: o que entra por uma ponta sai pela outra. Uma ponta fica dentro do contêiner (tipicamente chamada `eth0`) e a outra permanece no *namespace* do *host*, onde é conectada a uma ponte (*bridge*) ou tratada diretamente pelo plugin de rede (Linux Kernel Documentation, 2024).

Esse mecanismo pode ser reproduzido à mão, sem Docker nem Kubernetes. O bloco a seguir é **executável** e é a base do laboratório do capítulo 11:

Listing 8.1 – Emulação de um contêiner com network namespace e par veth (executável, requer root).

```
sudo ip netns add pod1
sudo ip link add veth-host type veth peer name veth-pod
sudo ip link set veth-pod netns pod1

sudo ip addr add 10.200.0.1/24 dev veth-host
sudo ip link set veth-host up
sudo ip netns exec pod1 ip addr add 10.200.0.2/24 dev veth-pod
```

```
sudo ip netns exec pod1 ip link set veth-pod up
sudo ip netns exec pod1 ip link set lo up

# teste de conectividade do "pod" ate o host
sudo ip netns exec pod1 ping -c 2 10.200.0.1

# limpeza (remove tambem o veth)
sudo ip netns del pod1
```

O ponto relevante para o eBPF é que a ponta `veth-host` é uma interface comum do *host*: é possível anexar a ela um programa XDP (no modo genérico) ou TC, e assim observar ou filtrar todo o tráfego que entra e sai do contêiner, sem tocar no seu interior. É isso que os plugins de rede baseados em eBPF fazem para cada *pod* (Cilium Authors, s.d.a).

O modelo de rede do Kubernetes

O Kubernetes define algumas exigências para a rede do cluster:

- cada *pod* recebe um endereço IP próprio, e todos os contêineres de um mesmo *pod* compartilham o mesmo *network namespace*;
- todo *pod* deve conseguir se comunicar com qualquer outro *pod* do cluster, sem NAT entre eles;
- *serviços* (`Service`) oferecem um endereço virtual estável que distribui as conexões entre as réplicas (*pods*) de uma aplicação.

O Kubernetes não implementa essa rede por conta própria; ele delega a um plugin (CNI, seção seguinte) e, para os serviços, a um componente chamado `kube-proxy` ou a uma alternativa. Cada uma dessas peças é um ponto onde o eBPF pode substituir mecanismos tradicionais.

Onde o eBPF se encaixa

A motivação mais citada para a substituição é a escala: com o *iptables*, o número de regras cresce com o número de serviços e *pods*, e o custo de atualizar e percorrer a lista cresce junto; consultas em mapas `HASH` do BPF têm custo aproximadamente constante (Cilium Authors, s.d.a; GREGG, 2019). Esse argumento é teórico neste manual: nenhuma medição comparativa foi feita aqui.

Tabela 1 – Mecanismos tradicionais de rede em Kubernetes e o papel do eBPF.

Função	Mecanismo tradicional	Alternativa com eBPF
Encaminhamento entre *pods*	*bridge* Linux, rotas, *overlay* VXLAN	programas TC/XDP na `veth` do *pod*, redirecionamento direto
Balanceamento de `Service`	regras *iptables* ou IPVS do `kube-proxy`	mapas BPF com os *backends* e seleção no kernel
Política de rede	regras *iptables* geradas a partir da `NetworkPolicy`	verificação por identidade em mapas BPF
Observabilidade	*tcpdump*, contadores de interface	programas BPF que emitem eventos de fluxo

8.2 CNI

Container Network Interface (CNI) é uma especificação que define como um *runtime* de contêineres configura a rede de um contêiner (Cloud Native Computing Foundation, s.d.). Quando o *kubelet* cria um *pod*, o *runtime* cria o *network namespace* e invoca o plugin CNI configurado no nó. O plugin é um executável que recebe, por variáveis de ambiente e por um JSON na entrada padrão, o comando (`ADD`, `DEL`, `CHECK`), o identificador do contêiner e o caminho do *namespace*, e devolve o resultado (endereço IP atribuído, rotas) (Cloud Native Computing Foundation, s.d.).

Na operação `ADD`, um plugin típico executa a sequência já vista no Listing 8.1: cria o par `veth`, move uma ponta para o *namespace* do *pod*, atribui um endereço IP e instala as rotas. Um plugin baseado em eBPF acrescenta um passo: **anexa um programa TC ou XDP à ponta do *host*** e registra o *pod* em mapas BPF (endereço, interface, identidade), de modo que o kernel já saiba tratá-lo no primeiro pacote.

A configuração do CNI de um nó fica, por convenção, em `/etc/cni/net.d/` e os binários dos plugins em `/opt/cni/bin/`. Os comandos a seguir apenas **inspecionam** um nó; eles são seguros, mas só produzem resultado em um nó Kubernetes real, portanto **não foram executados** nesta máquina:

Listing 8.2 – Inspeção da configuração CNI em um nó (comando de cluster, não executado).

```
ls /etc/cni/net.d/
cat /etc/cni/net.d/*.conf*
ls /opt/cni/bin/
```

A escolha do plugin determina o que é possível. Plugins simples, como `bridge` combinado com `host-local`, apenas conectam o *pod*; plugins mais completos, como o Cilium (capítulo 9), assumem também a política de rede, o balanceamento de serviços e a observabilidade usando eBPF (Cilium Authors, s.d.a). Trocar o CNI de um cluster existente é uma operação delicada, que normalmente exige reiniciar os *pods*; por isso, a escolha costuma ser feita na criação do cluster.

8.3 NETWORK POLICIES

Por padrão, o Kubernetes é permissivo: todo *pod* aceita tráfego de qualquer outro. Uma `NetworkPolicy` é um recurso que restringe esse comportamento, selecionando *pods* por rótulo e listando de quem eles podem receber tráfego (*ingress*) e para quem podem enviar (*egress*) (The Kubernetes Authors, s.d.).

Dois pontos sobre a semântica merecem destaque. Primeiro, a `NetworkPolicy` é apenas uma **declaração**: o objeto é armazenado no cluster, mas **quem a aplica é o plugin de rede**. Um CNI que não implementa políticas aceita o objeto e simplesmente o ignora (The Kubernetes Authors, s.d.). Segundo, a política é **aditiva**: a partir do momento em que algum objeto seleciona um *pod* para um sentido, o tráfego nesse sentido passa a ser negado, exceto o que alguma regra permita explicitamente.

O manifesto a seguir é um **exemplo de manifesto**, **não aplicado a um cluster**. Ele permite que apenas *pods* com o rótulo `app: web` acessem o banco de dados na porta 5432:

Listing 8.3 – NetworkPolicy que restringe o acesso ao banco de dados (exemplo de manifesto).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-permite-web
  namespace: loja
spec:
  podSelector:
    matchLabels:
      app: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: web
      ports:
        - protocol: TCP
          port: 5432
```

Uma prática comum é partir de uma política de **negação padrão** e liberar o necessário em seguida. A política vazia abaixo seleciona todos os *pods* do *namespace* e, por não listar nenhuma regra de entrada, nega todo o tráfego recebido:

Listing 8.4 – Negação padrão de entrada em um namespace (exemplo de manifesto).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
```

```
  name: negar-entrada
  namespace: loja
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

Como o eBPF aplica essas políticas. Um CNI baseado em eBPF não traduz a política em uma lista de regras de *iptables* por endereço. Em vez disso, ele atribui uma **identidade** numérica a cada conjunto de *pods* com os mesmos rótulos e grava no mapa BPF quais pares de identidades podem se comunicar. No caminho do pacote, o programa TC identifica a identidade da origem, consulta o mapa e decide entre aceitar e descartar (Cilium Authors, s.d.a). Como a regra refere-se a identidades e não a endereços, um *pod* que é recriado com outro IP continua coberto pela política sem reconfiguração. Esse é o mesmo padrão de lista de bloqueio em mapa do capítulo 7, com identidades no lugar de endereços.

Aplicar e consultar as políticas em um cluster faz-se com o `kubectl`:

Listing 8.5 – Aplicação e consulta de políticas com kubectl (comando de cluster, não executado).

```
kubectl apply -f db-permite-web.yaml
kubectl get networkpolicy -n loja
kubectl describe networkpolicy db-permite-web -n loja
```

8.4 COMUNICAÇÃO ENTRE PODS E SERVIÇOS

Pod para pod

Quando dois *pods* estão no mesmo nó, o tráfego percorre as duas pontas `veth` e o caminho local do *host*. Quando estão em nós diferentes, o pacote atravessa a rede entre nós, seja por roteamento direto, seja encapsulado em um túnel (*overlay*, como VXLAN ou Geneve). Com eBPF, o redirecionamento pode ser feito pelo próprio programa TC da `veth`, por meio de helpers como `bpf_redirect`, evitando a travessia da pilha de roteamento do *host* quando o destino é local (Linux Kernel Documentation, 2024; Cilium Authors, s.d.a).

Pod para serviço

Um `Service` do tipo `ClusterIP` possui um endereço virtual que **não pertence a nenhuma interface**. Para que a conexão chegue a uma réplica real, algum componente precisa substituir o endereço virtual pelo de um *pod backend*. No modelo tradicional, o `kube-proxy`

faz isso instalando regras *iptables* (ou IPVS) que realizam DNAT a cada pacote novo, e o *conntrack* garante que os pacotes seguintes da conexão sigam o mesmo caminho.

Uma implementação eBPF pode agir de duas formas (Cilium Authors, s.d.a):

- **Na camada de pacote**: um programa TC ou XDP traduz o endereço de destino consultando um mapa de serviços e reescreve os cabeçalhos, de forma análoga ao DNAT do *netfilter*, porém com tabelas indexadas;
- **Na camada de soquete**: um programa anexado ao *cgroup* do *pod*, nos ganchos de `connect()` (tipos `cgroup/connect4` e `cgroup/connect6`), troca o endereço do serviço pelo endereço de um *backend* **antes** de o pacote existir. A conexão já nasce destinada ao *pod* real, e nenhum pacote precisa ser reescrito no caminho.

A segunda abordagem elimina a tradução por pacote para o tráfego iniciado dentro do cluster. Ela é possível porque o gancho `cgroup/connect4` do eBPF permite que um programa altere o endereço de destino de uma chamada `connect()` no momento em que ela é feita (tipo de programa `BPF_PROG_TYPE_CGROUP_SOCK_ADDR`) (Linux Kernel Documentation, 2024). A *observação* que se obtém dessa abordagem é um efeito colateral importante: como a tradução ocorre antes do pacote, ferramentas como o *tcpdump* dentro do *pod* já enxergam o endereço do *backend*, e não o do serviço.

O programa de exemplo abaixo ilustra o formato desse tipo de gancho. Ele **é um esboço didático**: apenas redireciona conexões para um endereço fixo e não consulta mapa de serviços, balanceamento nem trata erros. Foi verificado apenas quanto à compilação.

Listing 8.6 – Esboço de redirecionamento no gancho cgroup/connect4 (didático).

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

#define SVC_IP    0x0a600001  /* 10.96.0.1, "ClusterIP" ficticio */
#define BACKEND   0x0ac80002  /* 10.200.0.2, "pod" ficticio */

SEC("cgroup/connect4")
int redireciona(struct bpf_sock_addr *ctx)
{
    if (ctx->user_ip4 == bpf_htonl(SVC_IP) &&
        ctx->user_port == bpf_htons(80)) {
        ctx->user_ip4 = bpf_htonl(BACKEND);
    }
    return 1;   /* 1 = permitir a chamada connect() */
}

char LICENSE[] SEC("license") = "GPL";
```

Tráfego externo e DNS

Serviços dos tipos `NodePort` e `LoadBalancer` expõem o serviço fora do cluster. O tráfego de entrada chega a um nó, que precisa escolher um *backend* (possivelmente em outro nó) e preservar, quando possível, o endereço de origem do cliente. O XDP é um bom ponto para essa decisão, pois age antes da pilha (Cilium Authors, s.d.a). A resolução de nomes (DNS) é um serviço do cluster como qualquer outro, e é nesse tráfego que muitas falhas de rede em Kubernetes se manifestam; a observação de consultas DNS com eBPF é abordada no capítulo 9 com o Hubble.

Depuração

Quando um *pod* não alcança um serviço, os comandos abaixo ajudam a localizar a camada com defeito. São **comandos de cluster**, não executados nesta máquina:

Listing 8.7 – Passos de depuração de conectividade em um cluster (não executado).

```
kubectl get svc,endpoints -n loja          # o servico tem backends?
kubectl get pods -o wide -n loja           # em que no e IP estao os
   ↪ pods?
kubectl exec -it POD -n loja -- nslookup db   # o DNS resolve o servico?
kubectl exec -it POD -n loja -- nc -zv db 5432  # a porta responde?
```

Se o serviço não tiver *endpoints*, o problema está nos rótulos ou na saúde dos *pods*, e não na rede. Se tiver, mas a conexão falhar, o suspeito passa a ser uma política de rede (Listing 8.3), cuja ação de descarte pode ser observada com as ferramentas do capítulo 9.

9 CILIUM E EBPF

O Cilium é um projeto de código aberto que implementa rede, segurança e observabilidade para Kubernetes usando eBPF como base (Cilium Authors, s.d.a). Ele é o exemplo mais completo do que o capítulo 8 descreveu em termos gerais: um plugin CNI que anexa programas eBPF a cada *pod*, substitui o `kube-proxy`, aplica políticas por identidade e expõe os fluxos observados por meio do Hubble (Cilium Authors, s.d.b).

Aviso de escopo. Assim como no capítulo anterior, este capítulo é **conceitual**. Não havia cluster Kubernetes, `kubectl`, `kind` nem a CLI do Cilium na máquina em que o manual foi escrito. Nenhum comando deste capítulo foi executado; as formas e as opções foram tiradas da documentação oficial (Cilium Authors, s.d.a; Cilium Authors, s.d.b) e podem mudar entre versões. Antes de usar, confira a documentação da versão instalada. Os manifestos YAML foram verificados apenas quanto à sintaxe.

9.1 CILIUM COMO CNI

Componentes

A arquitetura do Cilium tem poucas peças (Cilium Authors, s.d.a):

- o **agente** (`cilium-agent`), executado como `DaemonSet`, isto é, um por nó. Ele recebe do Kubernetes os *pods*, serviços e políticas, compila e carrega os programas eBPF e mantém os mapas BPF;
- o **plugin CNI**, invocado pelo *kubelet* a cada criação de *pod*, que aciona o agente (conforme a operação `ADD` da seção CNI do capítulo 8);
- o **operador**, responsável por tarefas de escopo de cluster, como a alocação de blocos de endereços;
- o **Hubble**, camada de observabilidade, apresentada adiante.

Instalação

A forma mais direta de instalar o Cilium em um cluster novo é a CLI do projeto. O bloco a seguir é um **comando de cluster, não executado**:

Listing 9.1 – Instalação e verificação do Cilium com a CLI (comando de cluster, não executado).

```
cilium install
cilium status --wait
```

Alternativamente, o Cilium é instalado com Helm, o que permite fixar a versão e ativar funcionalidades por parâmetros. Os valores abaixo ativam a substituição do `kube-proxy` e o Hubble; os nomes de parâmetros mudam entre versões, e o `kubeProxyReplacement` exige que o cluster tenha sido criado sem o `kube-proxy` ou que ele seja removido:

Listing 9.2 – Instalação via Helm com Hubble e substituição do kube-proxy (comando de cluster, não executado).

```
helm repo add cilium https://helm.cilium.io/
helm install cilium cilium/cilium --namespace kube-system \
     --set kubeProxyReplacement=true \
     --set hubble.relay.enabled=true \
     --set hubble.ui.enabled=true
```

O que o Cilium carrega no kernel

Em cada nó, o agente anexa programas eBPF principalmente nos pontos estudados nos capítulos 4 e 5: TC (*ingress* e *egress*) nas interfaces dos *pods* e do nó, XDP opcionalmente na interface externa para balanceamento, e programas de *cgroup* para a tradução de serviços no `connect()` (Listing 8.6) (Cilium Authors, s.d.a).

Esses programas são visíveis com as mesmas ferramentas do restante do manual. Os comandos a seguir são **executáveis em um nó** (com privilégio de administrador), mas só produzem resultado onde o Cilium estiver instalado; a saída, portanto, não é reproduzida:

Listing 9.3 – Inspeção dos programas do Cilium em um nó com bpftool (requer root em um nó com Cilium).

```
sudo bpftool prog show | grep -i cil
sudo bpftool net show
sudo bpftool map show | head
```

O `bpftool net show` lista os programas XDP e TC anexados a cada interface. Em um nó com Cilium, espera-se encontrar programas TC nas interfaces dos *pods* (nomes iniciados por `lxc`) e na interface do nó; a ferramenta confirma na prática o que a seção anterior descreveu, e é um bom exercício comparar o que ela lista com o que o capítulo 5 ensinou a anexar manualmente.

9.2 NETWORK POLICIES

O Cilium aceita a `NetworkPolicy` padrão do Kubernetes (Listing 8.3) e oferece o recurso próprio `CiliumNetworkPolicy`, mais expressivo (Cilium Authors, s.d.a). A diferença central está na forma como as políticas são aplicadas. O Cilium atribui a cada *pod* uma **identidade de segurança**, derivada do conjunto de rótulos relevantes, e a identidade viaja com o pacote (por exemplo, em um campo do encapsulamento ou consultada em mapa no destino). A regra de política, no fim, é uma entrada de mapa BPF do tipo *(identidade de origem, porta, protocolo)* $\rightarrow$ *permitir*, e não uma lista de endereços IP.

Duas consequências práticas merecem destaque: a política acompanha o *pod* quando ele é recriado com outro endereço, e a verificação tem custo praticamente independente do número de *pods* (Cilium Authors, s.d.a).

O manifesto a seguir é um **exemplo de manifesto**, **não aplicado**. Ele equivale ao Listing 8.3, mas acrescenta uma regra de camada 7: somente requisições `GET` em caminhos que começam por `/api/`.

Listing 9.4 – CiliumNetworkPolicy com regra HTTP na camada 7 (exemplo de manifesto).

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: api-somente-get
  namespace: loja
spec:
  endpointSelector:
    matchLabels:
      app: api
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: web
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
          rules:
            http:
              - method: "GET"
                path: "/api/.*"
```

A regra de camada 7 tem um custo que a de camada 3/4 não tem: o Cilium precisa **interpretar o protocolo de aplicação**, e para isso redireciona o tráfego afetado a um *proxy* (baseado em Envoy) em vez de decidir apenas no programa eBPF (Cilium Authors, s.d.a). É uma troca consciente entre expressividade e desempenho: use a regra de camada 7 apenas onde

a distinção por método ou caminho for realmente necessária.

Também é possível controlar a **saída** (*egress*) por nome de domínio, útil para permitir que uma aplicação acesse um serviço externo específico sem manter listas de IP. O Cilium resolve essa regra observando as respostas DNS do próprio *pod* (Cilium Authors, s.d.a):

Listing 9.5 – Política de saída por nome de domínio (exemplo de manifesto).

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: web-saida-dns
  namespace: loja
spec:
  endpointSelector:
    matchLabels:
      app: web
  egress:
    - toEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: kube-system
            k8s:k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: ANY
          rules:
            dns:
              - matchPattern: "*"
    - toFQDNs:
        - matchName: "api.exemplo.com.br"
      toPorts:
        - ports:
            - port: "443"
              protocol: TCP
```

A primeira regra de saída libera as consultas ao DNS do cluster, que é pré-requisito para a regra `toFQDNs` funcionar, já que é a observação dessas respostas que informa ao Cilium quais endereços correspondem ao nome. O domínio `api.exemplo.com.br` é fictício.

Aplicar e listar os recursos usa o `kubectl`:

Listing 9.6 – Aplicação das políticas do Cilium (comando de cluster, não executado).

```
kubectl apply -f api-somente-get.yaml
kubectl get cnp -n loja
```

9.3 OBSERVABILIDADE COM HUBBLE

O Hubble é a camada de observabilidade do Cilium. Ele aproveita o fato de que os programas eBPF do Cilium já veem cada pacote para emitir **eventos de fluxo**, que incluem origem e destino por identidade (nome do *pod*, *namespace*), protocolo, porta e, o ponto mais valioso, o **veredito** da política (`FORWARDED`, `DROPPED`) (Cilium Authors, s.d.b). Isso responde à pergunta que o capítulo 8 deixou aberta: qual política derrubou a conexão?

Cada agente mantém um histórico limitado dos fluxos do seu nó. O componente *Hubble Relay* agrega os agentes e oferece uma única API para o cluster inteiro; a CLI `hubble` e a interface web (Hubble UI) consomem essa API (Cilium Authors, s.d.b). O bloco a seguir ativa e usa o Hubble. É um **comando de cluster, não executado**:

Listing 9.7 – Ativação e consulta de fluxos com o Hubble (comando de cluster, não executado).

```
cilium hubble enable --ui
cilium hubble port-forward &

hubble status
hubble observe --namespace loja
hubble observe --namespace loja --verdict DROPPED --follow
hubble observe --to-pod loja/api --protocol http
cilium hubble ui
```

O segundo comando de `hubble observe` é o mais útil para depuração de política: ele acompanha em tempo real apenas os fluxos descartados. Cada linha traz a origem, o destino, o protocolo e o veredito, de modo que o *pod* bloqueado e a política responsável ficam evidentes. A saída é por *pod* (nomes de recurso do Kubernetes) e não por endereço IP, o que a torna legível em um ambiente onde os endereços mudam o tempo todo. Neste manual, não há saída real a mostrar.

O Hubble também gera métricas no formato Prometheus (contagem de fluxos por veredito, consultas DNS e requisições HTTP, entre outras), que podem ser coletadas e exibidas em Grafana como no capítulo 6 (Cilium Authors, s.d.b; Prometheus Authors, s.d.; Grafana Labs, s.d.). Assim, a cadeia de observabilidade do Manual 1 e do capítulo 6 se estende a nível de cluster.

9.4 SEGURANÇA DE REDE COM CILIUM

Além das políticas de rede, o Cilium oferece recursos de segurança que se apoiam no eBPF (Cilium Authors, s.d.a):

- **Criptografia transparente** entre nós, com WireGuard ou IPsec, ativada por parâmetros

de instalação, sem alterar as aplicações;

- **Políticas por identidade em vez de endereço**, o que reduz a superfície de erro de listas de IP desatualizadas;
- **Visibilidade de descartes** por meio do Hubble, que dá ao administrador uma trilha de auditoria de tráfego negado;
- **Integração com o Tetragon** (Isovalent, s.d.), projeto da mesma família, que acrescenta observabilidade e imposição em tempo de execução sobre processos e chamadas de sistema.

A ativação da criptografia com WireGuard é feita na instalação (comando de cluster, **não executado**; os nomes dos parâmetros devem ser conferidos na documentação da versão):

Listing 9.8 – Ativação da criptografia transparente com WireGuard (comando de cluster, não executado).

```
cilium install --set encryption.enabled=true --set encryption.type=
   ↪ wireguard
cilium status
```

Limites e cuidados. A segurança do Cilium vale tanto quanto a configuração: uma política *default-allow* (nenhuma política selecionando o *pod*) continua permissiva, e o Hubble só vê o que passa pelos programas do Cilium. O agente roda com privilégios elevados em cada nó e carrega código no kernel; seu comprometimento é grave, e o acesso ao cluster deve ser controlado de acordo. Por fim, a mesma regra de honestidade vale para o restante deste manual: o que foi descrito aqui representa a documentação oficial (Cilium Authors, s.d.a), não uma execução. Para validá-lo, o caminho mais curto é um cluster local de teste (como *kind* ou *minikube*) e o roteiro da documentação do projeto.

10 FERRAMENTAS DE EBPF NETWORKING

Os capítulos anteriores escreveram programas eBPF à mão. Na prática, boa parte do trabalho com rede começa em ferramentas prontas: para descobrir *o que está acontecendo* usa-se BCC ou *bpftrace*; para inspecionar *o que está carregado* usa-se *bpftool*; para entregar uma solução própria, usa-se *libbpf*. Este capítulo apresenta as quatro ferramentas com foco em rede e mostra como se encaixam, na mesma ordem em que costumam ser necessárias (GREGG, 2019; GREGG, 2020).

Sobre a validação. Os comandos de rede do BCC e do *bpftrace* citados aqui foram executados nas rodadas de teste do Manual 1 (pasta `testes/` e `testes/resultados/`), em Ubuntu 26.04 com kernel 7.0. O resultado dessa rodada é relevante para a escolha da ferramenta: nesse kernel, as versões em Python do BCC `tcpconnect-bpfcc`, `tcpretrans-bpfcc` e `tcplife-bpfcc` **falharam na compilação** contra os cabeçalhos do kernel, enquanto as versões `tcpconnect-libbpf` e `tcplife-libbpf` e os *one-liners* de *bpftrace* executaram. Em outro kernel ou distribuição, o resultado pode ser diferente. Os demais comandos desta seção não foram executados neste kernel, e o leitor deve conferi-los em seu ambiente.

10.1 BCC

O BCC (*BPF Compiler Collection*) é uma coleção de ferramentas escritas em Python cujo núcleo em C é compilado em tempo de execução com o LLVM, contra os cabeçalhos do kernel em uso (IO Visor Project, s.d.). Ele traz dezenas de utilitários de linha de comando prontos, entre eles os de rede mais usados. Nas distribuições Debian e Ubuntu, são instalados pelo pacote `bpfcc-tools` com o sufixo `-bpfcc` (por exemplo, `tcpconnect-bpfcc`); em outras distribuições, o nome é simplesmente `tcpconnect`.

Tabela 2 – Ferramentas do BCC voltadas a rede.

Ferramenta	O que mostra
`tcpconnect`	cada conexão TCP ativa (`connect()`) com PID, comando, endereços e porta
`tcpaccept`	cada conexão TCP passiva aceita (`accept()`)
`tcplife`	conexões TCP encerradas, com duração e bytes transferidos
`tcpretrans`	retransmissões TCP, com endereços e estado da conexão
`tcptop`	volume de bytes por conexão, em formato semelhante ao `top`

Todas exigem privilégio de administrador. Os exemplos abaixo usam o nome com sufixo do Ubuntu:

Listing 10.1 – Ferramentas de rede do BCC.

```
sudo tcpconnect-bpfcc            # novas conexoes de saida
```

```
sudo tcpconnect-bpfcc -p 1234     # apenas o PID 1234
sudo tcpaccept-bpfcc              # conexoes aceitas por servidores
   ↪ locais
sudo tcplife-bpfcc                # conexoes encerradas, com duracao e
   ↪ bytes
sudo tcpretrans-bpfcc             # retransmissoes
```

O campo mais valioso dessas saídas é o **PID e o nome do comando**: diferente do *tcpdump*, que enxerga pacotes, as ferramentas do BCC ligam a conexão ao processo que a criou, porque observam chamadas de função do kernel no contexto desse processo (GREGG, 2019). Para saber *quem* está falando com o exterior, é a informação decisiva.

Limitação. A compilação em tempo de execução exige o compilador LLVM e os cabeçalhos do kernel instalados no sistema, o que os torna pesados para servidores de produção e sensíveis a incompatibilidades entre versões de kernel, como a observada na rodada de testes do Manual 1. A alternativa moderna são as versões CO-RE do mesmo conjunto de ferramentas, descritas na seção sobre *libbpf*.

10.2 BPFTRACE

O *bpftrace* é uma linguagem de alto nível para rastreamento: o usuário escreve uma sonda e uma ação em poucas linhas, e a ferramenta compila e carrega o programa eBPF (bpftrace Contributors, s.d.). É o instrumento ideal para perguntas ad hoc, que ainda não justificam um programa C.

Todos os *one-liners* abaixo usam pontos de rastreamento estáveis do kernel (*tracepoints*) ou funções do TCP; os mesmos pontos já apareceram no capítulo 6:

Listing 10.2 – One-liners de rede em bpftrace.

```
# quem esta enviando dados por TCP (bytes por processo)
sudo bpftrace -e 'kprobe:tcp_sendmsg { @bytes[comm] = sum(arg2); }'

# retransmissoes TCP, com processo e portas
sudo bpftrace -e 'tracepoint:tcp:tcp_retransmit_skb {
    printf("%s sport=%d dport=%d\n", comm, args->sport, args->dport); }'

# transicoes de estado de conexoes TCP
sudo bpftrace -e 'tracepoint:sock:inet_sock_set_state {
    printf("%s %d -> %d sport=%d dport=%d\n", comm,
           args->oldstate, args->newstate, args->sport, args->dport); }'

# histograma de tamanho dos pacotes recebidos (tracepoint de rede
   ↪ generico)
sudo bpftrace -e 'tracepoint:net:netif_receive_skb { @len = hist(args->
```

```
   ↪ len); }'
```

Os três primeiros *one-liners* têm correspondentes executados com sucesso na rodada de testes do Manual 1 (`tcp_sendmsg sum`, `tcp_retransmit_skb` e `inet_sock_set_state`); o histograma sobre `netif_receive_skb` é novo neste manual e depende de o *tracepoint* existir no kernel em uso. Para conferir os pontos disponíveis, a listagem faz parte do próprio *bpftrace*:

Listing 10.3 – Descoberta de pontos de rastreamento de rede.

```
sudo bpftrace -l 'tracepoint:net:*'
sudo bpftrace -l 'tracepoint:tcp:*'
sudo bpftrace -l 'kprobe:tcp_*' | head
sudo bpftrace -lv 'tracepoint:tcp:tcp_retransmit_skb'
```

A opção `-lv` mostra os campos de `args`, o que evita adivinhar nomes como `sport` e `dport`. Esses campos pertencem à interface do *tracepoint* e podem variar entre versões do kernel.

Limitação. O *bpftrace* não anexa programas a XDP ou TC, que são os ganchos da *ação* sobre pacotes; ele serve para observar. Decisões de descartar, redirecionar ou modificar exigem programas escritos em C (capítulos 4 e 5) (Linux Kernel Documentation, 2024).

10.3 BPFTOOL

O *bpftool* é a ferramenta oficial de inspeção e administração do subsistema BPF, mantida junto ao código do kernel (Linux Kernel Documentation, 2024). Para rede, três subcomandos concentram o uso: `prog`, `map` e `net`.

Listing 10.4 – Inspeção de programas, mapas e ganchos de rede com bpftool.

```
sudo bpftool prog show                  # todos os programas carregados
sudo bpftool prog show type xdp         # apenas programas XDP
sudo bpftool prog dump xlated id 42     # codigo apos o verificador
sudo bpftool prog dump jited id 42      # codigo de maquina gerado

sudo bpftool map show                   # todos os mapas
sudo bpftool map dump id 7              # conteudo de um mapa

sudo bpftool net show                   # XDP, TC e outros anexos por
   ↪ interface
sudo bpftool net show dev eth0

sudo bpftool prog tracelog              # le a saida de bpf_printk (
   ↪ trace_pipe)
```

O `bpftool net show` responde a uma pergunta que nenhuma outra ferramenta res-

ponde com tanta clareza: **que programas eBPF estão ligados à rede deste sistema?** Ele lista, por interface, os programas XDP, TC e de *flow dissector* e, em seções separadas, os anexos de *cgroup*. É o primeiro comando a executar ao investigar um nó desconhecido, em especial um nó Kubernetes com Cilium (capítulo 9), e também a forma de conferir se um programa realmente foi desanexado depois de um experimento.

O `bpftool` também administra o ciclo de vida: carregar (`prog load`, `prog loadall`), anexar (`net attach`, `net detach`) e editar mapas (`map update`, `map delete`), como no Listing 7.4. E, com `btf dump` e `gen skeleton`, ele prepara o terreno para a *libbpf*:

Listing 10.5 – Geração de vmlinux.h e do esqueleto (skeleton) com bpftool.

```
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
bpftool gen skeleton fw_xdp.bpf.o > fw_xdp.skel.h
```

O segundo comando foi executado sobre o objeto do Listing 7.1 e gerou o cabeçalho sem erros; ambos dispensam privilégio de administrador, pois só leem arquivos.

10.4 LIBBPF

A *libbpf* é a biblioteca C oficial para carregar e gerenciar programas eBPF a partir do espaço de usuário (libbpf Contributors, s.d.; NAKRYIKO, 2020). O modelo recomendado é **CO-RE** (*Compile Once, Run Everywhere*): o programa é compilado uma vez, com informações de tipo BTF, e a biblioteca ajusta os acessos a estruturas do kernel no momento do carregamento, o que dispensa os cabeçalhos do kernel instalados no alvo (NAKRYIKO, 2020). Essa é a diferença essencial em relação ao BCC e a razão de as versões `-libbpf` das ferramentas terem funcionado onde as de Python falharam. Elas são empacotadas em Debian e Ubuntu como `libbpf-tools`:

Listing 10.6 – Ferramentas de rede da libbpf-tools.

```
sudo tcpconnect-libbpf
sudo tcplife-libbpf
```

Para uma solução própria, o fluxo é: escrever o programa (arquivo `.bpf.c`), compilar com `clang -target bpf`, gerar o esqueleto com `bpftool gen skeleton` e escrever um *carregador* em C que usa as funções geradas. O carregador a seguir completa o programa do Listing 7.1: carrega o objeto, preenche opcionalmente a lista de bloqueio com um IP recebido na linha de comando, anexa o XDP à interface e imprime os contadores a cada segundo.

Listing 10.7 – Carregador libbpf para o firewall XDP (fw_loader.c).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <arpa/inet.h>
```

```
#include <net/if.h>
#include <linux/if_link.h>
#include <bpf/libbpf.h>
#include <bpf/bpf.h>
#include "fw_xdp.skel.h"

static volatile sig_atomic_t parar;
static void sai(int s) { parar = 1; }

int main(int argc, char **argv)
{
    if (argc < 2) {
        fprintf(stderr, "uso: %s <interface> [ip_a_bloquear]\n", argv
   ↪ [0]);
        return 1;
    }
    int ifindex = if_nametoindex(argv[1]);
    if (!ifindex) { perror("if_nametoindex"); return 1; }

    struct fw_xdp_bpf *skel = fw_xdp_bpf__open_and_load();
    if (!skel) { fprintf(stderr, "falha ao carregar o programa\n");
   ↪ return 1; }

    if (argc > 2) {
        struct in_addr a;
        __u32 v = 1;
        if (inet_pton(AF_INET, argv[2], &a) != 1) {
            fprintf(stderr, "IP invalido\n");
            return 1;
        }
        /* a.s_addr ja esta em ordem de rede, como no pacote */
        bpf_map_update_elem(bpf_map__fd(skel->maps.blocklist),
                            &a.s_addr, &v, BPF_ANY);
    }

    struct bpf_link *link =
        bpf_program__attach_xdp(skel->progs.xdp_fw, ifindex);
    if (!link) { fprintf(stderr, "falha ao anexar o XDP\n"); return 1; }

    signal(SIGINT, sai);
    int ncpu = libbpf_num_possible_cpus();
    __u64 *vals = calloc(ncpu, sizeof(__u64));
    while (!parar) {
        sleep(1);
        for (__u32 k = 0; k < 2; k++) {
            __u64 soma = 0;
            if (bpf_map_lookup_elem(bpf_map__fd(skel->maps.stats),
                                    &k, vals) == 0)
```

```
                for (int i = 0; i < ncpu; i++)
                    soma += vals[i];
            printf("%s=%llu ", k ? "descartados" : "aceitos",
                   (unsigned long long)soma);
        }
        printf("\n");
    }
    bpf_link__destroy(link);
    fw_xdp_bpf__destroy(skel);
    return 0;
}
```

A compilação e a execução seguem:

Listing 10.8 – Compilação e execução do carregador.

```
clang -target bpf -O2 -g -I/usr/include/x86_64-linux-gnu \
      -c fw_xdp.bpf.c -o fw_xdp.bpf.o
bpftool gen skeleton fw_xdp.bpf.o > fw_xdp.skel.h
gcc -O2 -Wall -I. fw_loader.c -o fw_loader -lbpf -lelf -lz

sudo ./fw_loader eth0 10.0.0.2    # Ctrl-C desanexa o programa
```

Os três primeiros comandos (compilar o objeto, gerar o esqueleto e compilar o carregador) foram executados sem privilégio de administrador e terminaram sem erros nesta máquina. A **execução** do carregador, que carrega o programa no kernel, **ainda não foi realizada**. A anexação via `bpf_program__attach_xdp` usa o modo escolhido pelo kernel; para forçar o modo genérico em uma interface sem suporte nativo, a API `bpf_xdp_attach` aceita a flag `XDP_FLAGS_SKB_MODE`.

Quando usar cada ferramenta. A regra geral deste manual é: comece pelo mais simples que responde à pergunta. Uma pergunta pontual ("quem abriu essa conexão?") pede `tcpconnect` ou um *one-liner* de *bpftrace*. Uma pergunta sobre o estado do sistema ("que programas estão ligados à rede?") pede `bpftool net`. Uma solução que precisa durar, ser distribuída ou agir sobre pacotes pede um programa próprio com *libbpf*. O capítulo 11 aplica as quatro em três cenários.

11 EXEMPLOS PRÁTICOS

Este capítulo reúne três exemplos que combinam o que os capítulos anteriores apresentaram: captura e análise de pacotes, monitoramento de conexões e filtragem de tráfego. Todos rodam em um **laboratório local**, sem cluster e sem acesso à rede externa, montado com um *network namespace* e um par `veth` (Listing 8.1). Os comandos são executados por `testes/09-redes-exemp` e os programas C ficam em `testes/redes/`.

Estado da validação. Os três exemplos foram executados pelo script `testes/09-redes-exemplos` em Ubuntu 26.04 com kernel 7.0, em um laboratório com *network namespace* e par `veth`. O script terminou com 17 testes aprovados e nenhum reprovado, e sua saída está em `testes/resultados/09-r` O histograma de tamanhos de pacote e a perda artificial com `netem` foram executados pelo script `testes/10-pendencias.sh`, com a saída em `testes/resultados/10-pendencias.txt`.

Preparação do laboratório. O laboratório é o do Listing 8.1: o *namespace* `pod1` (endereço `10.200.0.2`) faz o papel de um contêiner, e a interface `veth-host` (`10.200.0.1`), no *host*, é onde os programas são anexados. Para os exemplos 1 e 3, repita os seis primeiros comandos daquele bloco (sem o `ip netns del`, que desfaz o laboratório); o `testes/09-redes-exemplos.sh` faz isso automaticamente, e remove tudo ao final.

11.1 CAPTURA E ANÁLISE DE PACOTES

Objetivo. Observar cada pacote que chega à interface `veth-host` e confrontar o resultado com o *tcpdump*, a referência clássica de captura.

O programa XDP abaixo registra, com `bpf_printk`, o protocolo, a origem e o tamanho de cada pacote IPv4 e, para TCP, as portas e a flag SYN. Retorna `XDP_PASS`, e portanto não altera o tráfego. A saída de `bpf_printk` vai para o *trace pipe* do kernel, o que é suficiente para um exemplo didático; para volumes reais, a alternativa é o `ringbuf` do Listing 5.2.

Listing 11.1 – Programa XDP que registra pacotes com bpf_printk (pkt_trace.bpf.c).

```
#include <linux/bpf.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/in.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

SEC("xdp")
int pkt_trace(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
```

```
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end || eth->h_proto != bpf_htons(
   ↪ ETH_P_IP))
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    bpf_printk("proto=%d src=%pI4 len=%d", ip->protocol, &ip->saddr,
               (int)(data_end - data));
    if (ip->protocol == IPPROTO_TCP) {
        struct tcphdr *tcp = (void *)ip + ip->ihl * 4;
        if ((void *)(tcp + 1) <= data_end)
            bpf_printk("tcp %d -> %d syn=%d", bpf_ntohs(tcp->source),
                       bpf_ntohs(tcp->dest), tcp->syn);
    }
    return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

A execução tem quatro passos: compilar, carregar e anexar, gerar tráfego e ler. O terminal A lê o *trace pipe*; o terminal B gera tráfego a partir do *namespace*:

Listing 11.2 – Carga, geração de tráfego e leitura do trace pipe.

```
clang -target bpf -O2 -g -I/usr/include/x86_64-linux-gnu \
      -c pkt_trace.bpf.c -o pkt_trace.bpf.o

sudo mkdir -p /sys/fs/bpf/redes
sudo bpftool prog loadall pkt_trace.bpf.o /sys/fs/bpf/redes/pt
sudo bpftool net attach xdpgeneric pinned /sys/fs/bpf/redes/pt/pkt_trace
   ↪  \
     dev veth-host

# terminal A: ler o que o programa registra
sudo bpftool prog tracelog

# terminal B: gerar trafego ICMP e TCP a partir do "pod"
sudo ip netns exec pod1 ping -c 3 10.200.0.1
sudo ip netns exec pod1 curl -m 2 http://10.200.0.1:8099/    # sem
   ↪ servidor: so o SYN

# terminal C: o mesmo trafego visto pelo tcpdump
sudo tcpdump -n -i veth-host icmp or tcp port 8099
```

```
# limpeza
sudo bpftool net detach xdpgeneric dev veth-host
sudo rm -r /sys/fs/bpf/redes
```

O que verificar. Cada pacote ICMP do `ping` deve gerar uma linha com `proto=1` no *tracelog* (o protocolo 1 é o ICMP), e o `curl` sem servidor deve gerar linhas com `syn=1` para a porta 8099. O *tcpdump* deve mostrar os mesmos pacotes, e o número de pacotes ICMP nas duas saídas deve coincidir (o *ping* gera uma requisição ICMP por segundo, três no total). Divergências possíveis e seu motivo: o XDP vê apenas tráfego de **entrada** na interface (aqui, as requisições vindas do *pod*), enquanto o *tcpdump* captura nos dois sentidos, portanto as respostas do *host* ao *ping* só aparecem no *tcpdump*.

Análise complementar com bpftrace. Para a distribuição dos tamanhos dos pacotes, sem escrever C, use o *tracepoint* genérico (Listing 10.2). Aqui, o interesse é o formato do resultado, um histograma em potências de dois:

Listing 11.3 – Histograma de tamanho de pacotes na veth-host.

```
sudo bpftrace -e 'tracepoint:net:netif_receive_skb /str(args->name) == "
   ↪ veth-host"/ { @len = hist(args->len); }'
```

Interrompa com `Ctrl-C` para imprimir o histograma. O filtro usa o campo `name` do *tracepoint*; como o conteúdo dos campos depende do kernel, confirme-o com `bpftrace -lv 'tracepoint:net:netif_receive_skb'`.

11.2 MONITORAMENTO DE CONEXÕES

Objetivo. Identificar quem abre conexões TCP, por quanto tempo elas duram e quanto tráfego carregam, associando cada uma a um processo — a pergunta que o *tcpdump* não responde.

O cenário é um servidor HTTP mínimo e alguns clientes. Os comandos abaixo sobem o servidor e, em outros terminais, as ferramentas do capítulo 10:

Listing 11.4 – Monitoramento de conexões TCP com ferramentas eBPF.

```
# terminal A: servidor de teste
python3 -m http.server 8099 --bind 127.0.0.1

# terminal B: conexoes novas (use tcpconnect-bpfcc se nao tiver libbpf-
   ↪ tools)
sudo tcpconnect-libbpf

# terminal C: conexoes encerradas, com duracao e bytes
sudo tcplife-libbpf
```

```
# terminal D: estados do socket via bpftrace (ESTABLISHED = 1)
sudo bpftrace -e 'tracepoint:sock:inet_sock_set_state /args->newstate ==
   ↪  1/ {
    printf("%s -> dport %d\n", comm, args->dport); }'

# terminal E: gerar conexoes
for i in 1 2 3; do curl -s -o /dev/null http://127.0.0.1:8099/; done

# referencia independente: o que o kernel ja conhece
ss -tni 'sport = :8099'
```

O que verificar. As três ferramentas devem registrar três conexões para a porta 8099, todas iniciadas pelo comando `curl`. O `tcpconnect` mostra o instante e o processo; o `tcplife` mostra a duração de cada uma depois que terminam; o *bpftrace* mostra a transição para `ESTABLISHED`, tanto do lado do cliente quanto do servidor, de modo que as linhas aparecem em dobro (uma com o processo `curl` e outra com o `python3`). O `ss` é a verificação cruzada, mas só mostra conexões que ainda existem no instante da consulta; as curtas do `curl` já terão terminado. Essa diferença — o eBPF registra **eventos**, o `ss` mostra o **estado atual** — é o motivo de as duas abordagens serem complementares.

Nas rodadas de teste do Manual 1, as versões `-libbpf` de `tcpconnect` e `tcplife` executaram, enquanto as versões `-bpfcc` falharam na compilação nesse kernel (capítulo 10); por isso o exemplo prefere as primeiras. Nos testes, com o servidor na porta 8099, o `tcpconnect-libbpf`, o `tcplife-libbpf` e o *one-liner* de `bpftrace` registraram a conexão.

Retransmissões. Quando uma conexão está lenta, as retransmissões são o primeiro suspeito. Em um laboratório, é possível provocá-las com perda artificial de pacotes usando o `tc netem` no *namespace*, e observá-las com o *tracepoint* `tcp_retransmit_skb`:

Listing 11.5 – Provocação e observação de retransmissões.

```
sudo ip netns exec pod1 tc qdisc add dev veth-pod root netem loss 20%
sudo bpftrace -e 'tracepoint:tcp:tcp_retransmit_skb {
    printf("%s sport=%d dport=%d\n", comm, args->sport, args->dport); }'
   ↪  &
sudo ip netns exec pod1 curl -m 5 http://10.200.0.1:8099/
sudo ip netns exec pod1 tc qdisc del dev veth-pod root
```

Aqui, o `netem` descarta 20% dos pacotes que saem do *namespace*. Para que o *pod* alcance o servidor, este deve escutar em `10.200.0.1` (por exemplo, `python3 -m http.server 8099 -bind 10.200.0.1`), e não só em `127.0.0.1`. O `netem` exige o módulo `sch_netem` no kernel, que pode estar ausente em configurações mínimas. A expectativa é ver linhas de retransmissão com as portas da conexão; o número exato é aleatório, e o campo `comm` pode mostrar um contexto do kernel em vez de `curl`, pois as retransmissões por temporizador ocorrem fora do contexto do processo. No teste, a retransmissão apareceu com `comm` igual a `swapper/7`, a tarefa ociosa da CPU 7.

11.3 FILTRAGEM DE TRÁFEGO

Objetivo. Bloquear e liberar uma origem em tempo de execução, sem recarregar o programa, e conferir o efeito pelos contadores do próprio mapa.

O programa é o *firewall* do Listing 7.1, anexado à `veth-host`. O *namespace* `pod1` é a origem a bloquear: a requisição ICMP do *ping* chega com origem `10.200.0.2`, o que corresponde a `0a c8 00 02` em bytes.

Listing 11.6 – Bloqueio e liberação de uma origem com o firewall XDP.

```
clang -target bpf -O2 -g -I/usr/include/x86_64-linux-gnu \
      -c fw_xdp.bpf.c -o fw_xdp.bpf.o

sudo mkdir -p /sys/fs/bpf/redes
sudo bpftool prog loadall fw_xdp.bpf.o /sys/fs/bpf/redes/fw \
     pinmaps /sys/fs/bpf/redes/fw/maps
sudo bpftool net attach xdpgeneric pinned /sys/fs/bpf/redes/fw/xdp_fw \
     dev veth-host

# 1. sem regras: o ping deve funcionar
sudo ip netns exec pod1 ping -c 2 -W 2 10.200.0.1

# 2. bloquear 10.200.0.2: o ping deve falhar (100% de perda)
sudo bpftool map update pinned /sys/fs/bpf/redes/fw/maps/blocklist \
     key hex 0a c8 00 02 value hex 01 00 00 00
sudo ip netns exec pod1 ping -c 2 -W 2 10.200.0.1

# 3. conferir os contadores (indice 0 = aceitos, 1 = descartados)
sudo bpftool map dump pinned /sys/fs/bpf/redes/fw/maps/stats

# 4. liberar: o ping deve voltar a funcionar
sudo bpftool map delete pinned /sys/fs/bpf/redes/fw/maps/blocklist \
     key hex 0a c8 00 02
sudo ip netns exec pod1 ping -c 2 -W 2 10.200.0.1

# limpeza
sudo bpftool net detach xdpgeneric dev veth-host
sudo rm -r /sys/fs/bpf/redes
```

O que verificar. O passo 1 deve resultar em 0% de perda, o passo 2 em 100% de perda e o passo 4 novamente em 0%. No passo 3, o contador de índice 1 (descartados) deve ser maior que zero, sendo a soma dos valores por CPU. O ponto central do exemplo é o passo 2: o programa não foi recarregado; mudou-se apenas o conteúdo de um mapa.

Variações. Três mudanças simples, usando o mesmo laboratório, exploram o restante do capítulo 7:

- bloquear uma **porta** em vez de um IP, preenchendo o mapa `blocked_ports` (Listing 7.4) e testando com um servidor na porta bloqueada;
- trocar o programa por `syn_guard.bpf.c` (Listing 7.8) e disparar muitos SYNs com um laço de `curl` ou `nc`, comparando o contador `descartados` com o limite de 100 por segundo;
- usar `tc_egress_fw.bpf.c` (Listing 7.6) no *namespace*, para impedir que o *pod* abra conexões em uma porta de destino.

Cada variação compila (ver o script de teste), mas nenhuma foi executada.

Carregador em C. O Listing 10.7 faz os passos 1 e 2 em um único programa. O script de teste o chama por três segundos com o endereço do *pod* como argumento e verifica se os contadores foram impressos.

Resumo do laboratório. Os três exemplos exercitam, em sequência, a observação passiva (XDP com `XDP_PASS`), a observação de eventos (`tcpconnect`, `tcplife`, *bpftrace*) e a ação (`XDP_DROP` com regras em mapas). É a mesma progressão que o capítulo 7 recomenda para qualquer política: observar primeiro, impor depois.

12 CONCLUSÃO

Este manual percorreu a rede do Linux sob a ótica do eBPF. Os capítulos 3 a 5 estabeleceram a base: onde a pilha de rede oferece ganchos, o que o XDP e o TC podem ver e fazer em cada um deles, e como mapas BPF levam informação e configuração entre o kernel e o espaço de usuário. O capítulo 6 usou essa base para observar tráfego sem alterá-lo, e o capítulo 7 a usou para agir sobre ele, com listas de bloqueio em mapas e a limitação de SYN por origem. A progressão se resume em uma regra prática que se repete ao longo do texto: observar primeiro, impor depois.

Os capítulos 8 e 9 levaram as mesmas ideias a contêineres e ao Kubernetes, onde a identidade de uma carga de trabalho substitui o endereço IP como critério de política, e o Cilium e o Hubble fazem em escala de cluster o que os exemplos de laboratório fazem em uma interface. O capítulo 10 mostrou o conjunto de ferramentas (BCC, *bpftrace*, *bpftool* e *libbpf*) e o capítulo 11 as reuniu em três cenários executáveis. Vale reforçar a distinção feita em cada capítulo: o que é conceitual (Kubernetes e Cilium, sem cluster disponível) e o que é executável (o restante), e, entre os executáveis, o que foi executado no laboratório (captura, conexões e filtragem XDP) e o que foi apenas compilado.

O caminho natural para fechar essa lacuna está na pasta `testes/` do repositório. O script `09-redes-exemplos.sh` executa os exemplos do capítulo 11 em um laboratório com *network namespace*. Em Ubuntu 26.04 com kernel 7.0, ele terminou com 17 testes aprovados, e a saída guardada em `testes/resultados/` é a evidência dos exemplos de captura, conexões e filtragem. Ficam sem validação os capítulos de Kubernetes e Cilium e o tráfego de teste no limitador de SYN, que só teve o carregamento aceito. Para o leitor que deseja avançar, os próximos passos são validar esses trechos, em especial os de Kubernetes e Cilium em um cluster local de teste, ampliar o *firewall* para IPv6 e rastreamento de conexões, e conectar a telemetria de rede à cadeia de Prometheus e Grafana apresentada no Manual 1.

REFERÊNCIAS

ALVES, R. C.; PLINTA, J. P. S. H. **Monitoramento de Funções Virtualizadas de Redes: Um Estudo Comparativo entre Ferramentas de Extração de Comportamento**. Pinhais: Instituto Federal do Paraná, 2026. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação).

bpftrace Contributors. **bpftrace: High-level tracing language for Linux**. s.d. Disponível em: <https://github.com/bpftrace/bpftrace>. Acesso em: 6 out. 2026.

CASSAGNES, C. *et al.* The rise of eBPF for non-intrusive performance monitoring. In: **NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium**. [S.l.]: IEEE Press, 2020. p. 1–7. Disponível em: <https://doi.org/10.1109/NOMS47738.2020.9110434>.

Cilium Authors. **Cilium Documentation**. s.d. Disponível em: <https://docs.cilium.io>. Acesso em: 9 nov. 2026.

____. **Network, Service & Security Observability for Kubernetes (Hubble)**. s.d. Disponível em: <https://docs.cilium.io/en/stable/observability/hubble/>. Acesso em: 9 nov. 2026.

Cloud Native Computing Foundation. **Container Network Interface (CNI) Specification**. s.d. Disponível em: <https://github.com/containernetworking/cni/blob/main/SPEC.md>. Acesso em: 9 nov. 2026.

Cloudflare. **ebpf_exporter: Prometheus exporter for custom eBPF metrics and OpenTelemetry traces**. s.d. Disponível em: <https://github.com/cloudflare/ebpf_exporter>. Acesso em: 6 out. 2026.

ETSI Industry Specification Group (ISG) NFV. **Network Functions Virtualisation (NFV); Architectural Framework**. Sophia Antipolis, 2014. Disponível em: <https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/01.02.01_60/gs_NFV002v010201p.pdf>.

GODARD, S.; Sysstat Contributors. **sysstat: System performance monitor for Linux**. 2024. Disponível em: <https://github.com/sysstat/sysstat>.

Grafana Labs. **Grafana documentation**. s.d. Disponível em: <https://grafana.com/docs/grafana/latest/>. Acesso em: 6 out. 2026.

GREGG, B. **BPF Performance Tools**. Boston: Addison-Wesley, 2019.

____. **Systems Performance: Enterprise and the Cloud**. 2. ed. Boston: Addison-Wesley, 2020.

IO Visor Project. **BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more**. s.d. Disponível em: <https://github.com/iovisor/bcc>. Acesso em: 6 out. 2026.

Isovalent. **Tetragon: eBPF-based Security Observability and Runtime Enforcement**. s.d. Disponível em: <https://tetragon.io/docs/>. Acesso em: 6 out. 2026.

libbpf Contributors. **libbpf: Automated upstream mirror for libbpf stand-alone build**. s.d. Disponível em: <https://github.com/libbpf/libbpf>. Acesso em: 6 out. 2026.

Linux Kernel Documentation. **BPF Documentation**. 2024. Disponível em: <https://www.kernel.org/doc/html/latest/bpf/>.

Linux man-pages project. **proc(5) — Linux manual page**. 2024. Disponível em: <https://man7.org/linux/man-pages/man5/proc.5.html>.

MCCANNE, S.; JACOBSON, V. The BSD packet filter: A new architecture for user-level packet capture. In: **Proceedings of the USENIX Winter 1993 Conference**. San Diego: [s.n.], 1993. p. 259–270.

NAKRYIKO, A. **BPF CO-RE (Compile Once – Run Everywhere)**. 2020. Disponível em: <https://nakryiko.com/posts/bpf-portability-and-co-re/>.

PLINTA, J. P.; NAHAS, B. R. **ebpf-scitec: minicurso introdutório de eBPF**. 2026. Repositório de código. Disponível em: <https://github.com/joaopedroplinta/ebpf-scitec>.

Prometheus Authors. **Prometheus: Overview**. s.d. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 6 out. 2026.

RESEARCH, D. P. **Passive TCP RTT Latency Monitoring and Aggregation in the Linux Kernel using epping and eBPF**. [S.l.], 2024.

The Falco Authors. **Falco: Cloud Native Runtime Security**. s.d. Disponível em: <https://falco.org/docs/>. Acesso em: 6 out. 2026.

The Kubernetes Authors. **Network Policies**. s.d. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. Acesso em: 9 nov. 2026.

VENÂNCIO, G. *et al.* Beyond vnfm: Filling the gaps of the etsi vnf manager to fully support vnf life cycle operations. **International Journal of Network Management**, v. 31, n. 5, set. 2021. Disponível em: <https://ideas.repec.org/a/wly/intnem/v31y2021i5ne2068.html>.